

HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FALTY OF COMPUTER SCIENCE AND ENGINEERING



CAPSTONE PROJECT COMPUTER ENGINEERING

Motion Planning around Obstacles

Semester 251

Instructor: Assoc. Prof. Le Hong Trang

STT	Full name	ID	Note
1	Nguyen Tran Dang Khoa	2211635	
2	Ho Dang Khoa	2211588	

Ho Chi Minh City, December 2025

Tóm tắt

Hoạch định chuyển động cho robot trong môi trường chứa vật cản vốn là một bài toán tối ưu hóa phi tuyến đầy thách thức do tính chất không lồi của không gian cấu hình. Trong khi các phương pháp dựa trên lấy mẫu truyền thống thường gặp hạn chế về tính tối ưu và khả năng xử lý các ràng buộc động lực học phức tạp, bài báo này đề xuất một phương pháp tiếp cận mới dựa trên Đồ thị các Tập Lồi (Graphs of Convex Sets - GCS). Bằng cách phân rã không gian tự do thành các vùng lồi hữu hạn và mô hình hóa quỹ đạo robot sử dụng đường cong Bézier, chúng tôi thiết lập bài toán dưới dạng tối ưu hóa hỗn hợp nguyên. Cuối cùng ta áp dụng kỹ thuật nới lỏng lồi (convex relaxation) chặt chẽ để chuyển đổi bài toán phức tạp này thành bài toán Lập trình nón bậc hai (Second-Order Cone Programming - SOCP) dễ giải. Kết quả thực nghiệm trên các hệ thống có số chiều cao chứng minh phương pháp này vượt trội hơn so với các thuật toán PRM truyền thống, giảm thiểu đáng kể thời gian tính toán trong khi vẫn đảm bảo tìm được quỹ đạo tối ưu toàn cục. Chúng tôi kiểm chứng GCS trên một loạt các môi trường với độ phức tạp tăng dần, từ các vật cản 2D đơn giản và mê cung phức tạp đến các thiết bị bay quadrotor động lực học và tay máy 7 bậc tự do (7-DoF) trong không gian nhiều chiều. Các thực nghiệm số chứng minh rằng GCS đạt hiệu năng vượt trội một cách nhất quán so với các bộ hoạch định dựa trên lấy mẫu được sử dụng rộng rãi (ví dụ: PRM). Cụ thể, trong các nhiệm vụ nhiều chiều, GCS giảm thời gian truy vấn trực tuyến lên đến một bậc so với PRM tiêu chuẩn, đồng thời tìm ra các quỹ đạo tối ưu toàn cục ngắn hơn đáng kể (giảm tới 40% độ dài) so với các bộ hoạch định dựa trên lấy mẫu đã được xử lý hậu kỳ [1].

Contents

1	Giới thiệu	1
1.1	Động lực và Thách thức Kỹ thuật	1
1.2	Problem statement	3
1.3	Goals	5
1.4	Scope	5
1.5	Cấu trúc luận văn	5
2	Kiến thức nền tảng	6
2.1	Phân tích Lỗi và Tối ưu hóa	6
2.1.1	Tập lỗi	6
2.1.2	Hàm lỗi	7
2.1.3	Tối ưu hóa lỗi	8
2.1.4	Tối ưu hóa nón	9
2.1.5	Đồng nhất hóa	10
2.1.6	Bài toán Nguyên Hỗn hợp	11
2.2	Lý thuyết Đồ thị và Bài toán Đường đi Ngắn nhất	12
2.2.1	Cơ sở Lý thuyết Đồ thị	12
2.2.1.1	Các định nghĩa và khái niệm cơ bản . .	12
2.2.1.2	Tối ưu hóa tổ hợp trên đồ thị	13
2.2.2	Bài toán Đường đi Ngắn nhất (SPP)	14
2.2.2.1	Định nghĩa và các thuật toán cổ điển . .	14
2.2.2.2	Mô hình dòng mạng cho Bài toán Đường đi Ngắn nhất	15

2.3	Sinh quỹ đạo Động lực học (Kinodynamic)	16
2.3.1	Mô hình hóa Kinodynamic	16
2.3.2	Các ràng buộc về độ trơn và tính liên tục	17
2.3.3	Cơ sở lý thuyết về đường cong Bézier	18
2.4	Phân hoạch Lỗi	20
2.4.1	Định nghĩa	20
2.4.2	Phân loại Phương pháp Phân hoạch Lỗi	21
2.4.2.1	Phân hoạch Lỗi Chính xác (Exact Convex Decomposition – ECD)	21
2.4.2.2	Phân hoạch Lỗi Xấp xỉ (Approximate Convex Decomposition – ACD)	22
3	Related works	23
3.1	Sampling-Based Methods	23
3.2	Discussion	23
4	Proposed Solution	24
4.1	Phân hoạch không gian an toàn	24
4.2	Các Phương pháp Cốt lõi cho Phân hoạch Không gian Tự do	25
4.2.1	Phân hoạch Lỗi Xấp xỉ (ACD) của Đa giác	25
4.2.1.1	Các Khái niệm Cơ bản trong Phân rã Lỗi	25
4.2.1.2	Ý tưởng và Chi tiết Thuật toán	26
4.2.1.3	Các Thước đo Độ lõm	28
4.2.1.4	Đặc tính của Thuật toán	29
4.2.2	Iterative Regional Inflation by Semi-Definite Programming (IRIS)	29
4.2.2.1	Ý tưởng và Chi tiết Thuật toán	29
4.2.2.2	Đặc tính của Thuật toán	35

4.2.3	Gieo mầm Tự động qua Lớp phủ Clique Khả kiến (VCC)	36
4.2.3.1	Ý tưởng và Chi tiết Thuật toán	37
4.2.3.2	Đặc tính và Tính tối ưu	40
4.3	Đồ thị các tập lỗi	40
4.3.1	Mô hình bài toán Đường đi ngắn nhất trong Graph of Convex Sets	40
4.3.2	Phân tích độ phức tạp	42
4.3.3	Mô hình chi tiết các đỉnh và cạnh	43
4.3.4	Đặc tả hàm chi phí	44
4.3.5	Mô hình Mixed-Integer Convex Programming thông qua Perspective Relaxation	45
4.4	Proposed solution	47
4.5	Experiment setup	47
4.5.1	Implementation details	47
4.6	Results and analysis	47
5	Conclusion	48
5.1	Summary	48
5.2	Future Work	48
	References	49
	Appendix A ...	51
A.1		51
A.2		51

List of Figures

Figure 1.1	Boston Dynamics' Atlas and Amazon's Robin	2
Figure 2.1	Ví dụ về tập lỗi	6
Figure 2.2	Ví dụ về bao lỗi	7
Figure 4.1	(a) Nếu $x \in \partial P_i > 0$, hàm Resolve gộp biên ∂P_i vào đa giác P_0 . (b) Nếu $x \in \partial P_0$, hàm Resolve tách đa giác P thành hai đa giác P_1 và P_2 . (c) Độ lõm tại điểm x thay đổi sau khi đa giác được phân rã.	27
Figure 4.2	Quá trình đệ quy tiếp tục cho đến khi đạt đến giới hạn độ lõm đầu vào của người dùng τ	27
Figure 4.3	Giả sử r là điểm lõm có độ lõm lớn nhất. Sau khi giải quyết (resolve) r , độ lõm của s tăng lên. Nếu $\text{concave}(r) < \tau$, vùng s sẽ không bao giờ được xử lý, ngay cả khi $\text{concave}(s)$ lớn hơn τ	28
Figure 4.4	Độ lõm SL có thể xử lý được túi trong (a) một cách chính xác vì không có hướng chuẩn nào của các đỉnh trong túi ngược với hướng chuẩn của cầu. Tuy nhiên, túi trong (b) có thể dẫn đến độ lõm giảm không đơn điệu.	29
Figure 4.5	Siêu phẳng Phân cách	30
Figure 4.6	Ellipsoid Nội tiếp Cực đại	33

Figure 4.7	Lắp lại	34
Figure 4.8	Độ phức tạp thời gian [10]	35
Figure 4.9	Lấy mẫu & Xây dựng Đồ thị Khả kiến (Visibility Graphs)	37
Figure 4.10	Tìm Lớp phủ Clique	38
Figure 4.11	Khởi tạo Ellipsoid	38
Figure 4.12	Lấp đầy thành Đa diện	39
Figure 4.13	Lắp lại	39

List of Tables

Chapter 1

Giới thiệu

Trong chương 1, tổng quan, mục tiêu và các mục đích của đề tài nghiên cứu được trình bày. Bố cục của báo cáo cũng được giới thiệu.

1.1 Động lực và Thách thức Kỹ thuật

Việc đưa ra các quyết định tối ưu trong thời gian thực là một nền tảng cốt lõi của kỹ thuật hiện đại. Trong nhiều hệ thống phức tạp, đặc biệt là trong lĩnh vực robot tiên tiến, yêu cầu kết hợp chặt chẽ giữa **các quyết định rời rạc** và **điều khiển liên tục** trở nên hết sức rõ ràng.

- **Robot hình người (ví dụ: Atlas của Boston Dynamics):** Việc di chuyển trên địa hình phức tạp, chẳng hạn như các bài toán parkour, đòi hỏi robot phải đồng thời lựa chọn vị trí đặt chân (quyết định rời rạc) và tính toán quỹ đạo chuyển động cho trọng tâm cũng như các chi (điều khiển liên tục). Hiện nay, nhiều phương pháp vẫn phụ thuộc vào việc “lập trình thủ công” các quyết định rời rạc, điều này làm giảm đáng kể mức độ tự chủ của robot khi hoạt động trong những môi trường mới hoặc không xác định [2].
- **Robot công nghiệp (ví dụ: Robin của Amazon):** Hiệu quả của quá trình phân loại hàng hóa phụ thuộc vào việc tối ưu đồng thời thứ tự lựa chọn các thùng chứa (rời rạc) và quỹ đạo chuyển động của

cánh tay robot (liên tục) [3]. Chỉ cần cải thiện 1% tốc độ vận hành thông qua tối ưu hóa tốt hơn cũng có thể tương đương với việc xử lý thêm hàng triệu kiện hàng mỗi năm.

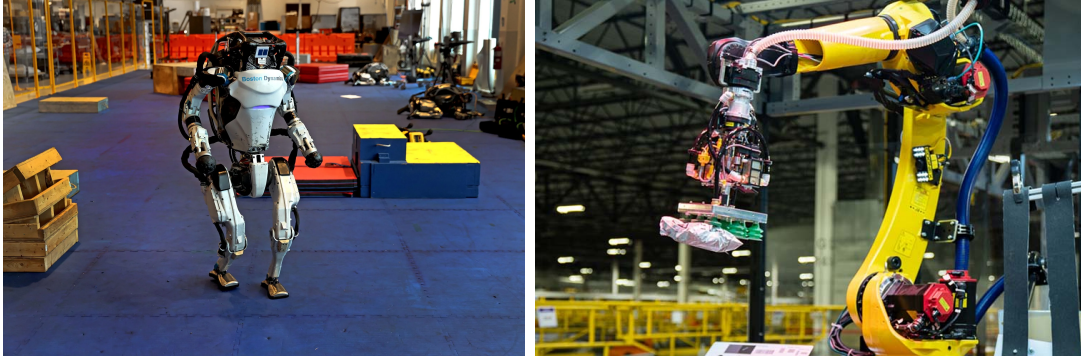


Figure 1.1: ***Bên trái:** Robot Atlas của Boston Dynamics thực hiện parkour, yêu cầu lập kế hoạch bước chân rời rạc và tối ưu quỹ đạo liên tục. **Bên phải:** Cánh tay robot Robin của Amazon phân loại hàng hóa, trong đó việc lựa chọn thùng chứa rời rạc và quỹ đạo chuyển động liên tục cần được tối ưu đồng thời.*

Tuy nhiên, việc giải quyết đồng thời các bài toán kết hợp này vẫn là một thách thức mở. Trong thực tế, các phương pháp hiện nay thường phải dựa vào sự can thiệp thủ công của con người hoặc chấp nhận các nghiệm dưới mức tối ưu.

Về mặt thuật toán, việc lựa chọn phương pháp lập kế hoạch chuyển động buộc các nhà nghiên cứu phải đánh đổi giữa nhiều yếu tố mâu thuẫn nhau, bao gồm số chiều của không gian trạng thái, các ràng buộc động học–động lực học và tính đầy đủ của thuật toán.

- **Tối ưu hóa quỹ đạo:** Các phương pháp này có ưu thế trong việc xử lý động lực học của robot và các bài toán có không gian trạng thái nhiều chiều. Tuy nhiên, khi môi trường chứa nhiều vật cản khiến bài toán trở nên phi lồi, chúng thường dễ rơi vào **cực tiểu cục bộ**, dẫn đến việc không tìm được quỹ đạo tránh va chạm [4].
- **Các phương pháp lập kế hoạch dựa trên lấy mẫu:** Để khắc phục hạn chế trên, cộng đồng robot học thường sử dụng các phương

pháp lấy mẫu như RRT hoặc PRM nhờ vào tính “đầy đủ theo xác suất” (probabilistic completeness). Tuy nhiên, cái giá phải trả là các quỹ đạo thu được thường **kém tối ưu**, và việc áp đặt các **ràng buộc vi phân liên tục** lên các mẫu rời rạc vẫn là một bài toán rất khó LLaValle1998, [5].

Lập trình lồi hỗn hợp số nguyên (Mixed-Integer Convex Programming – MICP) đã nổi lên như một hướng tiếp cận đầy hứa hẹn nhằm thu hẹp khoảng cách giữa hai nhóm phương pháp trên. MICP kết hợp được tính đầy đủ của các thuật toán dựa trên lấy mẫu với khả năng xử lý động lực học của tối ưu hóa quỹ đạo, đồng thời đảm bảo **tính tối ưu toàn cục** trong một khuôn khổ thống nhất. Tuy vậy, việc ứng dụng MICP trong thực tế vẫn bị hạn chế nghiêm trọng bởi chi phí tính toán rất cao, khi ngay cả những bài toán quy mô nhỏ cũng có thể mất tới vài phút để giải.

Mục tiêu của nghiên cứu này là phá vỡ rào cản tính toán đó. Chúng tôi tập trung vào một lớp bài toán lập kế hoạch chuyển động quan trọng có chứa các ràng buộc vi phân, và đề xuất một bộ lập kế hoạch dựa trên MICP có khả năng giải quyết các bài toán không gian nhiều chiều một cách đáng tin cậy chỉ trong vài giây, thông qua việc giải **một bài toán lồi duy nhất**.

1.2 Problem statement

In the ever-evolving world of technology, the concept of an e-commerce website employing a recommender system to customize the user experience is not unfamiliar, particularly in the gadget and computer sector. Despite the prevalence of e-commerce platforms as standard marketing and sales channels for these firms, there is a need to improve the current methods of user interaction, specifically in facilitating users to effectively retrieve their desired products. While these websites offer a wide range of products,

the existing user interface and navigation system present challenges for customers to quickly and efficiently find specific computers and gadgets, resulting in a subpar user experience.

The current user interaction design of e-commerce websites in the computer and gadgets sector lacks the necessary features and functionalities to enhance the browsing and search experience. Users often struggle to locate their desired products due to limited search filters, inadequate categorization, and a lack of intuitive navigation options. This leads to frustration, increased browsing time, and a potential loss of interest in exploring the full range of offerings.

One of the specific challenges lies in the use of filters to narrow down search results, which often requires users to possess a certain level of domain knowledge in gadgets, electronics, or computers. This can cause confusion for users who are new to this field and may not be familiar with the specific terminology or technical specifications. Similarly, searching using full-text search poses difficulties, as users need to have a good understanding of how to formulate effective search queries to retrieve products that meet their needs.

In summary, by empowering users to find products that meet their needs without requiring extensive technical expertise or struggling to formulate what they need in proper words, the websites can attract a broader customer base, drive conversion rates, and establish a competitive advantage in the market.

1.3 Goals

1.4 Scope

1.5 Cấu trúc luận văn

Luận văn này được tổ chức thành năm chương, cụ thể như sau:

- **Chương 1** trình bày tổng quan về đề tài, bao gồm động lực nghiên cứu, mục tiêu cũng như phạm vi của luận văn.
- **Chương 2** tập trung giới thiệu các kiến thức nền tảng cần thiết cho việc hiểu và phát triển phương pháp đề xuất, bao gồm tối ưu hóa lồi, tối ưu hóa hỗn hợp số nguyên và lý thuyết đồ thị.
- **Chương 3** tổng hợp và phân tích các công trình nghiên cứu liên quan nhằm làm rõ bối cảnh nghiên cứu, cũng như những hạn chế còn tồn tại của các phương pháp hiện nay.
- **Chương 4** trình bày khuôn khổ GCS và cách xây dựng bài toán MICP được đề xuất. Trên cơ sở đó, chúng tôi đưa ra các kết quả thực nghiệm và phân tích hiệu quả của phương pháp trong nhiều môi trường khác nhau với mức độ phức tạp tăng dần.
- **Chương 5** tổng kết toàn bộ nội dung của luận văn và đề xuất các hướng phát triển trong tương lai.

Chapter 2

Kiến thức nền tảng

Trong Chương 2, các kiến thức nền tảng trong đề tài được trình bày.

2.1 Phân tích Lỗi và Tối ưu hóa

2.1.1 Tập lồi

Trước khi thảo luận về các bài toán tối ưu, chúng tôi trình bày ngắn gọn các đối tượng hình học cơ bản được sử dụng trong luận văn này. Một tập $C \subseteq \mathbb{R}^n$ được gọi là *lồi* nếu đoạn thẳng nối giữa hai điểm bất kỳ trong tập đó nằm hoàn toàn bên trong tập.

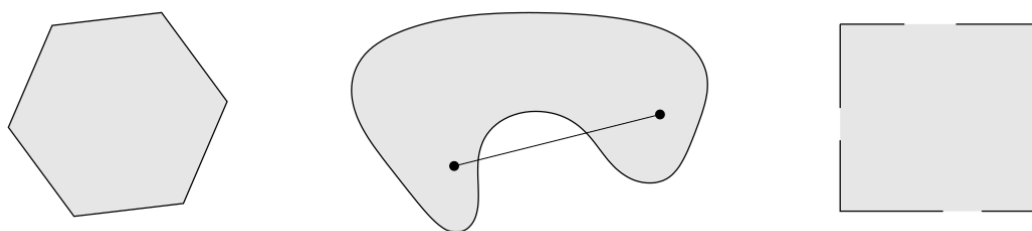


Figure 2.1: Các ví dụ về tập lồi và không lồi đơn giản. Trái: Lục giác, bao gồm cả biên, là một tập lồi. Giữa: Tập có dạng quả thận là không lồi, vì đoạn thẳng nối hai điểm nằm ngoài tập. Phải: Hình vuông chỉ chứa một số điểm biên mà không chứa tất cả, do đó không phải là tập lồi.[6]

Bao lồi (convex hull) là tập lồi nhỏ nhất chứa một tập điểm cho trước.

Một cách hình thức, bao lồi của tập các điểm $\{x_1, x_2, \dots, x_m\} \subseteq \mathbb{R}^n$ được định nghĩa như sau:

$$\text{conv}(\{x_1, x_2, \dots, x_m\}) = \left\{ \sum_{i=1}^m \lambda_i x_i \mid \lambda_i \geq 0, \sum_{i=1}^m \lambda_i = 1 \right\}.$$

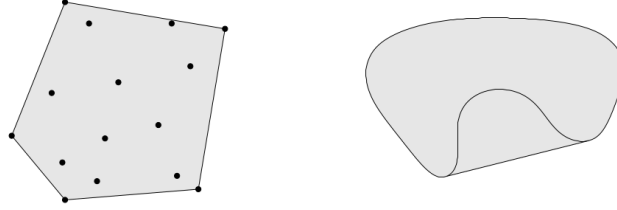


Figure 2.2: Bao lồi của một tập điểm (vùng tô đậm).[6]

Trong phạm vi của luận văn này, chúng tôi chủ yếu tập trung vào hai loại tập lồi:

- **Đa diện (Polyhedra):** Được định nghĩa là giao của một số hữu hạn các nửa không gian, $\mathcal{P} = \{x \in \mathbb{R}^n \mid Ax \leq b\}$. Các đa diện bị chặn được gọi là *đa diện lồi hữu hạn* (polytopes).
- **Ellipsoid:** Được định nghĩa là ảnh của một quả cầu đơn vị qua một phép biến đổi afin, thường được biểu diễn dưới dạng $\mathcal{E} = \{x \in \mathbb{R}^n \mid (x - c)^\top Q^{-1}(x - c) \leq 1\}$, với $Q \succ 0$.

Các tập này sẽ đóng vai trò là các miền (đỉnh) trong khuôn khổ lập kế hoạch chuyển động của chúng tôi.

2.1.2 Hàm lồi

Một hàm $f: \mathcal{D} \rightarrow \mathbb{R}$ được gọi là *lồi* nếu miền xác định $\mathcal{D} \subseteq \mathbb{R}^n$ là một tập lồi và nếu, với mọi $x, y \in \mathcal{D}$ và $\theta \in [0, 1]$, bất đẳng thức

$$f(\theta x + (1 - \theta)y) \leq \theta f(x) + (1 - \theta)f(y)$$

được thỏa mãn. Về mặt hình học, điều kiện này ngụ ý rằng đoạn thẳng nối hai điểm bất kỳ trên đồ thị của hàm nằm phía trên hoặc trùng với đồ thị đó. Một hàm được gọi là lồi nghiêm ngặt nếu bất đẳng thức trên là nghiêm với mọi $x \neq y$ và $\theta \in (0, 1)$. Ngược lại, một hàm f được gọi là lõm nếu $-f$ là hàm lồi.

Các hàm lồi sở hữu những tính chất giải tích mang tính nền tảng đối với tối ưu hóa hiệu quả. Quan trọng nhất trong số đó là mọi nghiệm tối ưu cục bộ của một hàm lồi đều được đảm bảo là nghiệm tối ưu toàn cục.

2.1.3 Tối ưu hóa lồi

Một **Bài toán Tối ưu Lồi (Convex Program – CP)** có dạng:

$$\text{tối thiểu hóa } f(x) \tag{2.1a}$$

$$\text{với điều kiện } x \in \mathcal{C}, \tag{2.1b}$$

trong đó hàm mục tiêu $f : \mathbb{R}^n \rightarrow \mathbb{R}$ và tập ràng buộc $\mathcal{C} \subset \mathbb{R}^n$ đều là lồi. Đôi khi, tập $\mathcal{C}$ được gọi là *tập khả thi* của CP, và các phần tử của nó được gọi là *nghiệm khả thi*. CP được gọi là *khả thi* nếu tồn tại ít nhất một nghiệm khả thi, tức là $\mathcal{C} \neq \emptyset$. Một nghiệm khả thi x^{opt} được gọi là *tối ưu* nếu $f(x^{\text{opt}}) \leq f(x)$ với mọi $x \in \mathcal{C}$. Nếu tồn tại nghiệm tối ưu, ta gọi $f^{\text{opt}} := f(x^{\text{opt}})$ là *giá trị tối ưu* của CP.

Một tính chất cơ bản của các CP là mọi nghiệm tối ưu cục bộ (tức là mọi điểm x^{opt} sao cho $f(x^{\text{opt}}) \leq f(x)$ với mọi x trong một lân cận của x^{opt}) cũng đồng thời là nghiệm tối ưu theo nghĩa toàn cục đã nêu [6]. Một giả thiết thường được thỏa mãn để đảm bảo sự tồn tại của nghiệm tối ưu là tập khả thi $\mathcal{C}$ không rỗng và compact.

Các CP là một lớp bài toán tối ưu nền tảng, với ứng dụng trong hầu như mọi lĩnh vực kỹ thuật. Phần lớn các CP gặp trong thực tế có thể được giải một cách hiệu quả và tin cậy bằng các phương pháp điểm trong (interior-

point methods) hoặc các thuật toán chuyên biệt khác như phương pháp đơn hình. Tuy nhiên, một cách chính xác thì không phải mọi CP đều có thể giải hiệu quả; ví dụ, tập các đa thức không âm là một nón lồi (trong không gian các hệ số đa thức), nhưng việc kiểm tra một đa thức có không âm hay không lại là một bài toán NP-hard. Trong luận văn này, chúng tôi sẽ có phần không quá chặt chẽ và sử dụng thuật ngữ “bài toán tối ưu lồi” gần như đồng nghĩa với “bài toán tối ưu có thể giải hiệu quả”.

2.1.4 Tối ưu hóa nón

Một **Bài toán Tối ưu Nón (Conic Program – KP)** là một CP với hàm mục tiêu tuyến tính và tập ràng buộc ở dạng nón:

$$\text{tối thiểu hóa} \quad c^\top x \tag{2.2a}$$

$$\text{với điều kiện} \quad Ax + b \in \mathcal{K}, \tag{2.2b}$$

trong đó $c \in \mathbb{R}^n$, $A \in \mathbb{R}^{m \times n}$, $b \in \mathbb{R}^m$, và $\mathcal{K} \subseteq \mathbb{R}^m$ là một nón lồi đóng. Các lớp KP đặc biệt bao gồm:

- **Bài toán Quy hoạch Tuyến tính (LP)** khi $\mathcal{K}$ là trục giao phần tư không âm.
- **Bài toán Quy hoạch Nón Bậc Hai (SOCP)** khi $\mathcal{K}$ là tích Descartes của các nón bậc hai.
- **Bài toán Quy hoạch Bán xác định (SDP)** khi $\mathcal{K}$ là (đồng cấu với) nón bán xác định.

Các lớp bài toán này có khả năng mô hình hóa tăng dần: mọi LP đều là một SOCP, và mọi SOCP đều là một SDP. Các SDP có thể được giải hiệu quả và bao phủ phần lớn các ứng dụng thực tiễn của tối ưu hóa lồi.

Một lớp CP quan trọng khác là **Bài toán Quy hoạch Toàn phương (QP)**, tức là các CP có dạng (2.1) với hàm mục tiêu toàn phương và tập

ràng buộc là một đa diện. Mọi QP đều có thể được biểu diễn lại dưới dạng SOCP. Tuy nhiên, trong nhiều trường hợp, các thuật toán active-set chuyên biệt cho lớp bài toán này có thể hiệu quả hơn so với việc sử dụng một bộ giải SOCP tổng quát.

2.1.5 Đồng nhất hóa

Đồng nhất hóa (homogenization) là một kỹ thuật trong phân tích lỗi nhằm biến đổi một tập không lỗi thành một tập lỗi bằng cách đưa vào một chiều bổ sung. Quá trình này đặc biệt hữu ích trong các bài toán tối ưu, nơi tính lỗi là điều kiện mong muốn để áp dụng các phương pháp giải hiệu quả.

Cho một tập $\mathcal{S} \subseteq \mathbb{R}^n$, đồng nhất hóa của $\mathcal{S}$, ký hiệu là $\tilde{\mathcal{S}}$, được định nghĩa như sau:

$$\tilde{\mathcal{S}} := \{(x, y) \in \mathbb{R}^{n+1} : y > 0, x \in y\mathcal{S}\}.$$

Phép biến đổi này thực chất “nâng” tập $\mathcal{S}$ lên một không gian có số chiều cao hơn, trong đó chiều bổ sung y cho phép biểu diễn các tổ hợp lỗi của các điểm trong $\mathcal{S}$. Tập đã đồng nhất hóa $\tilde{\mathcal{S}}$ là lỗi khi và chỉ khi tập ban đầu $\mathcal{S}$ là lỗi.

Đồng nhất hóa các tập ở dạng nón

Việc đồng nhất hóa một tập lỗi $\mathcal{C}$ được mô tả dưới dạng nón có thể được thực hiện rất dễ dàng. Thật vậy, với $y > 0$, ta có

$$y\mathcal{C} = \{yx : Ax + b \in \mathcal{K}\} = \{yx : A(yx) + by \in y\mathcal{K}\} = \{x' : Ax' + by \in \mathcal{K}\},$$

từ đó suy ra

$$\tilde{\mathcal{C}} = \{(x, y) : y > 0, Ax + by \in \mathcal{K}\}. \quad (2.3)$$

Ngoài ra, có thể dễ dàng kiểm chứng rằng

$$\text{cl}(\tilde{\mathcal{C}}) = \{(x, y) : y \geq 0, Ax + by \in \mathcal{K}\}. \quad (2.4)$$

Các biểu thức (2.3) và (2.4) có ý nghĩa thực tiễn rất lớn. Một cách khái quát, chúng cho thấy rằng nếu ta có thể thực hiện các phép tính hiệu quả với một tập $\mathcal{C}$ được mô tả dưới dạng nón, thì điều tương tự cũng đúng đối với (bao đóng của) tập đồng nhất hóa $\tilde{\mathcal{C}}$.

Đồng nhất hóa đặc biệt hữu ích trong các bài toán tối ưu, vì nó cho phép áp dụng các kỹ thuật tối ưu hóa lồi cho những bài toán ban đầu có vẻ không lồi. Bằng cách làm việc trong không gian đã được đồng nhất hóa, ta có thể khai thác các tính chất của tập và hàm lồi để tìm nghiệm tối ưu một cách hiệu quả hơn.

2.1.6 Bài toán Nguyên Hỗn hợp

Trong lĩnh vực tối ưu toán học, **Bài toán Nguyên Hỗn hợp** (Mixed-Integer Programs – MIPs) là một lớp bài toán trong đó vectơ biến quyết định không đồng nhất về miền giá trị. Cụ thể, các biến được chia thành hai nhóm: một nhóm bị ràng buộc nhận giá trị nguyên, và nhóm còn lại được phép nhận các giá trị thực liên tục. Một MIP có thể được phát biểu như sau:

$$\text{tối thiểu hóa} \quad f(x, z) \tag{2.5a}$$

$$\text{với điều kiện} \quad (x, z) \in \mathcal{C}, \tag{2.5b}$$

$$z \in \mathbb{Z}^m, \tag{2.5c}$$

trong đó $x \in \mathbb{R}^n$ biểu thị các biến liên tục, $z \in \mathbb{Z}^m$ biểu thị các biến nguyên, $f : \mathbb{R}^{n+m} \rightarrow \mathbb{R}$ là hàm mục tiêu, và $\mathcal{C} \subseteq \mathbb{R}^{n+m}$ là tập khả thi.

Trong khi các bài toán quy hoạch tuyến tính được xác định trên các đa diện lồi, cho phép áp dụng các phương pháp dựa trên gradient hoặc phương pháp đơn hình, việc áp đặt các ràng buộc nguyên khiến tập khả thi trở nên rời rạc và không lồi. Sự mất đi tính lồi này làm gia tăng đáng kể độ phức tạp tính toán, thường biến các bài toán có thể giải trong thời gian đa

thức thành các bài toán NP-hard. Do đó, các phương pháp phân tích tiêu chuẩn dựa trên tính liên tục và khả vi không còn áp dụng trực tiếp được. Để giải quyết các MIP, các phương pháp thư giãn thường được sử dụng. Cách tiếp cận phổ biến nhất là thư giãn quy hoạch tuyến tính, trong đó các ràng buộc nguyên $z_j \in \mathbb{Z}$ được thay thế bằng các ràng buộc liên tục yếu hơn (ví dụ, thay $z_j \in \{0,1\}$ bằng $0 \leq z_j \leq 1$). Giá trị hàm mục tiêu thu được từ bài toán đã thư giãn cung cấp một cận (cụ thể là cận dưới đối với bài toán tối thiểu hóa) cho giá trị tối ưu của MIP ban đầu. Nếu nghiệm của bài toán thư giãn thỏa mãn tất cả các ràng buộc nguyên, thì nó cũng chính là nghiệm tối ưu của MIP. Tuy nhiên, nếu nghiệm thư giãn chứa các giá trị phân số đối với những biến yêu cầu phải là số nguyên, các thuật toán như Branch-and-Bound thường được áp dụng. Phương pháp Branch-and-Bound chia nhỏ miền khả thi thành các miền con (các nhánh) bằng cách bổ sung các ràng buộc mới nhằm loại bỏ các giá trị phân số, từ đó tạo ra một cây tìm kiếm để khảo sát một cách có hệ thống không gian nghiệm.

2.2 Lý thuyết Đồ thị và Bài toán Đường đi Ngắn nhất

2.2.1 Cơ sở Lý thuyết Đồ thị

2.2.1.1 Các định nghĩa và khái niệm cơ bản

Một cách hình thức, một đồ thị được định nghĩa là một cặp có thứ tự $G = (\mathcal{V}, \mathcal{E})$, bao gồm một tập các đỉnh $\mathcal{V}$ và một tập các cạnh $\mathcal{E}$, trong đó mỗi cạnh nối một cặp đỉnh. Việc phân loại cấu trúc của đồ thị phụ thuộc vào tính có thứ tự của các cặp này: các cặp không có thứ tự $\{u, w\}$ biểu diễn đồ thị vô hướng, trong khi các cặp có thứ tự (u, w) đặc trưng cho đồ thị có hướng. Trong bối cảnh tối ưu hóa, đồ thị thường được mở rộng bằng

một hàm chi phí $c : \mathcal{E} \rightarrow \mathbb{R}$, gán một trọng số thực cho mỗi cạnh.

Các tính chất tô pô quan trọng bao gồm bậc của đỉnh, là số cạnh liên thuộc với đỉnh đó; trong đồ thị có hướng, khái niệm này được phân biệt thành bậc vào ($\deg^-$) và bậc ra ($\deg^+$). Tính liên thông được mô tả thông qua các đường đi—là các dãy hữu hạn gồm các đỉnh và cạnh phân biệt—trong đó độ dài đường đi được xác định bằng tổng chi phí của các cạnh cấu thành. Một chu trình được định nghĩa là một đường đi bắt đầu và kết thúc tại cùng một đỉnh. Các bài toán tối ưu thường được xét trên một miền tìm kiếm được xác định bởi các đồ thị con $H = (W, F)$, hình thành từ các tập con của đỉnh $W \subseteq \mathcal{V}$ và các cạnh $F \subseteq \mathcal{E}$, đồng thời bảo toàn các quan hệ liên thuộc.

2.2.1.2 Tối ưu hóa tổ hợp trên đồ thị

Các bài toán tối ưu hóa tổ hợp trên đồ thị thường nhằm tìm một cấu trúc con sao cho hàm mục tiêu đạt giá trị nhỏ nhất, đồng thời thỏa mãn các ràng buộc hợp lệ. Với một đồ thị có trọng số $G = (\mathcal{V}, \mathcal{E})$ và một tập các đồ thị con khả thi $\mathcal{H}$, bài toán là xác định một đồ thị con $H = (W, F) \in \mathcal{H}$ sao cho tổng trọng số của các thành phần của nó là nhỏ nhất. Bài toán này được biểu diễn hình thức như sau:

$$\text{tối thiểu hóa} \quad \sum_{v \in W} c_v + \sum_{e \in F} c_e \quad (1.1a)$$

$$\text{với điều kiện} \quad H = (W, F) \in \mathcal{H}. \quad (1.1b)$$

Để có thể áp dụng các kỹ thuật lập trình toán học tiêu chuẩn, chẳng hạn như Branch-and-Bound, bài toán được chuyển đổi sang dạng Quy hoạch Tuyến tính Nguyên (Integer Linear Programming – ILP). Quá trình này bao gồm việc tham số hóa không gian tìm kiếm bằng các vectơ liên thuộc $\mathbf{y}^H \in \{0, 1\}^{|\mathcal{V} \cup \mathcal{E}|}$, trong đó các biến nhị phân biểu thị việc một đỉnh hoặc một cạnh cụ thể có được chọn vào nghiệm hay không. Về mặt hình học,

tập các đồ thị con hợp lệ tương ứng với một đa diện $\mathcal{Y}$ trong không gian Euclid. Giao của đa diện này với lưới số nguyên, $\mathcal{Y} \cap \{0, 1\}^{|\mathcal{V} \cup \mathcal{E}|}$, tạo thành các vectơ liên thuộc của các đồ thị con khả thi. Do đó, bài toán tối ưu rời rạc ban đầu được tái biểu diễn thành một bài toán tối ưu tuyến tính trên miền nguyên:

$$\text{tối thiểu hóa} \quad \sum_{v \in \mathcal{V}} c_v y_v + \sum_{e \in \mathcal{E}} c_e y_e \quad (1.2a)$$

$$\text{với điều kiện} \quad \mathbf{y} \in \mathcal{Y} \cap \{0, 1\}^{|\mathcal{V} \cup \mathcal{E}|}. \quad (1.2b)$$

Cách phát biểu này cho phép áp dụng các công cụ của phân tích lỗi, đồng thời đảm bảo rằng nghiệm tối ưu của mô hình đại số tương ứng chính xác với cấu trúc con tối ưu của bài toán đồ thị ban đầu.

2.2.2 Bài toán Đường đi Ngắn nhất (SPP)

2.2.2.1 Định nghĩa và các thuật toán cổ điển

Bài toán Đường đi Ngắn nhất (Shortest Path Problem – SPP) trên một đồ thị có trọng số $G = (\mathcal{V}, \mathcal{E})$ nhằm xác định một đường đi P từ đỉnh nguồn s đến đỉnh đích t sao cho tổng trọng số các cạnh là nhỏ nhất. Nếu P bao gồm một dãy các cạnh $e_1, \dots, e_k$, mục tiêu là tối thiểu hóa $\sum_{i=1}^k c_{e_i}$.

Các phương pháp cổ điển để giải SPP phụ thuộc vào đặc tính của trọng số cạnh. Thuật toán Dijkstra sử dụng chiến lược tham lam và phù hợp với các đồ thị có trọng số không âm ($c_e \geq 0$). Đối với các đồ thị có cạnh mang trọng số âm, miễn là không tồn tại chu trình âm, thuật toán Bellman–Ford được sử dụng, mặc dù có độ phức tạp tính toán cao hơn. Thuật toán Floyd–Warshall mở rộng bài toán này cho trường hợp tìm đường đi ngắn nhất giữa mọi cặp đỉnh, cho phép tồn tại trọng số âm nhưng vẫn nhạy cảm với sự xuất hiện của các chu trình âm.

2.2.2.2 Mô hình dòng mạng cho Bài toán Đường đi Ngắn nhất

Bài toán Đường đi Ngắn nhất trên một đồ thị có hướng $G = (\mathcal{V}, \mathcal{E})$ với chi phí cạnh không âm $c_{uv} \geq 0$ có thể được mô hình hóa một cách chặt chẽ như một bài toán Dòng Chi phí Tối thiểu với một đơn vị dòng. Ta định nghĩa các biến quyết định nhị phân $x_{uv} \in \{0, 1\}$ cho mỗi cạnh $(u, v) \in \mathcal{E}$, trong đó $x_{uv} = 1$ cho biết cạnh đó thuộc đường đi tối ưu. Mô hình tối ưu được phát biểu như sau:

$$\text{minimize} \quad \sum_{(u,v) \in \mathcal{E}} c_{uv} x_{uv} \quad (2.6a)$$

$$\text{subject to} \quad \sum_{v \in \mathcal{V}^+(u)} x_{uv} - \sum_{v \in \mathcal{V}^-(u)} x_{vu} = \begin{cases} 1 & \text{if } u = s \\ -1 & \text{if } u = t \\ 0 & \text{otherwise} \end{cases}, \quad \forall u \in \mathcal{V} \quad (2.6b)$$

$$\sum_{v \in \mathcal{V}^-(u)} x_{vu} \leq 1, \quad \forall u \in \mathcal{V} \setminus \{s, t\} \quad (2.6c)$$

$$x_{uv} \geq 0, \quad \forall (u, v) \in \mathcal{E} \quad (2.6d)$$

Ở đây, $\mathcal{V}^+(u)$ và $\mathcal{V}^-(u)$ lần lượt ký hiệu tập các đỉnh kế tiếp (successors) và các đỉnh tiền nhiệm (predecessors) của đỉnh u . Các ràng buộc trong mô hình này đảm bảo những tính chất cấu trúc cụ thể. Phương trình (2.6b) biểu diễn điều kiện bảo toàn dòng (định luật Kirchhoff), đảm bảo rằng một đơn vị dòng rời khỏi đỉnh nguồn s , đi vào đỉnh đích t , và được bảo toàn tại tất cả các đỉnh trung gian. Ràng buộc (2.6c) áp đặt giới hạn dung lượng nút, giới hạn dòng đi vào mỗi đỉnh trung gian không vượt quá một đơn vị, qua đó đảm bảo cấu trúc đường đi đơn, trong đó không có đỉnh nào bị ghé thăm nhiều hơn một lần. Với chi phí không âm, các chu trình luôn là không tối ưu, do đó không cần thiết phải bổ sung các ràng buộc loại bỏ chu trình một cách tường minh.

Một ưu điểm quan trọng của cách mô hình hóa này nằm ở các tính chất

đại số của ma trận ràng buộc thu được từ (2.6b). Ma trận liên thuộc đỉnh–cạnh này có tính *hoàn toàn đơn mô* (Total Unimodularity – TUM). Theo lý thuyết tối ưu hóa tổ hợp, nếu ma trận ràng buộc là TUM và vectơ vế phải là nguyên, thì mọi nghiệm khả thi cơ sở của bài toán thư giãn Quy hoạch Tuyến tính (LP) đều được đảm bảo là nguyên [7]. Do đó, mặc dù chỉ áp đặt các ràng buộc không âm liên tục trong (2.6d), bài toán vẫn có thể được giải hiệu quả bằng các thuật toán LP tiêu chuẩn để thu được một đường đi nhị phân hợp lệ, mà không cần đến các phương pháp lập trình nguyên có chi phí tính toán cao.

2.3 Sinh quỹ đạo Động lực học (Kinodynamic)

Mục này trình bày các công thức toán học để sinh ra các quỹ đạo khả thi về mặt động lực học (kinodynamically feasible). Khác với lập kế hoạch đường đi hình học, vốn chỉ xem xét các cấu hình không va chạm trong không gian làm việc, lập kế hoạch chuyển động (kinodynamic) tính đến các ràng buộc vi phân của hệ thống, chẳng hạn như các giới hạn về vận tốc, gia tốc và độ giật (jerk). Chúng tôi tận dụng các đường cong Bézier làm cơ chế tham số hóa quỹ đạo, phương pháp này đặc biệt thuận lợi cho khuôn khổ Đồ thị các Tập lỗi (GCS) nhờ vào các tính chất lỗi của nó.

2.3.1 Mô hình hóa Kinodynamic

Xét một hệ thống robot với sự tiến triển trạng thái được chi phối bởi một tập hợp các phương trình vi phân thường (ODEs):

$$\dot{\mathbf{x}}(t) = f(\mathbf{x}(t), \mathbf{u}(t)), \quad \mathbf{x}(t) \in \mathcal{X}, \mathbf{u}(t) \in \mathcal{U} \quad (2.7)$$

trong đó $\mathbf{x}(t) \in \mathbb{R}^n$ đại diện cho vector trạng thái (thường bao gồm vị trí, vận tốc và gia tốc), và $\mathbf{u}(t) \in \mathbb{R}^m$ biểu thị các tín hiệu điều khiển đầu vào. Mục tiêu là tính toán một quỹ đạo $\sigma(t) : [0, T] \rightarrow \mathcal{X}$ sao cho tối thiểu hóa một phiếm hàm chi phí cụ thể trong khi thỏa mãn:

1. **Điều kiện biên:** Các trạng thái khởi đầu và đích đến $\mathbf{x}(0) = \mathbf{x}_{start}$ và $\mathbf{x}(T) = \mathbf{x}_{goal}$.
2. **Ràng buộc an toàn:** $\sigma(t) \in \mathcal{X}_{free}$ với mọi t , đảm bảo tránh va chạm.
3. **Giới hạn động lực học:** Các ràng buộc bất đẳng thức trên các đạo hàm bậc cao nhằm đảm bảo tính khả thi vật lý:

$$|\dot{\sigma}(t)| \leq v_{max}, \quad |\ddot{\sigma}(t)| \leq a_{max}, \quad |\dddot{\sigma}(t)| \leq j_{max} \quad (2.8)$$

2.3.2 Các ràng buộc về độ trơn và tính liên tục

Trong khuôn khổ GCS, một quỹ đạo hoàn chỉnh được cấu thành từ nhiều đoạn Bézier ghép nối (piecewise). Để đảm bảo chuyển động trơn tru qua các ranh giới của các tập lỗi kề nhau, các ràng buộc liên tục phải được áp đặt tại các điểm chuyển tiếp. Gọi $r_A(t)$ được xác định bởi các điểm điều khiển $\{a_0, \dots, a_n\}$ và $r_B(t)$ được xác định bởi $\{b_0, \dots, b_n\}$ là hai đoạn liên tiếp.

- **Liên tục C^0 (Vị trí):** Đảm bảo đường đi được kết nối liền mạch.

$$a_n = b_0 \quad (2.9)$$

- **Liên tục C^1 (Vận tốc):** Đảm bảo vận tốc liên tục, ngăn chặn các xung gia tốc vô hạn.

$$a_n - a_{n-1} = b_1 - b_0 \quad (2.10)$$

- **Liên tục C^2 (Gia tốc):** Đảm bảo việc tác động lực/mô-men xoắn

diễn ra liên tục, điều này rất quan trọng để giảm thiểu hao mòn cơ khí và độ giật (jerk).

$$(a_n - 2a_{n-1} + a_{n-2}) = (b_2 - 2b_1 + b_0) \quad (2.11)$$

Bằng cách áp đặt các ràng buộc đẳng thức này tại các cạnh của đồ thị, chúng ta tạo ra một quỹ đạo trơn toàn cục tuân thủ các đặc tính vật lý cơ bản của robot.

2.3.3 Cơ sở lý thuyết về đường cong Bézier

Để giải quyết bài toán lập kế hoạch kinodynamic bằng phương pháp số, việc tham số hóa các hàm quỹ đạo thông qua một số hữu hạn các biến quyết định là rất cần thiết. Để đạt được mục đích này, chúng tôi sử dụng các đường cong Bézier. Phần này nhắc lại định nghĩa và các tính chất cơ bản của họ đường cong này. Một đường cong Bézier được xây dựng dựa trên các đa thức Bernstein. Đa thức Bernstein thứ k bậc d , với $k = 0, \dots, d$, được định nghĩa là:

$$\beta_{k,d}(s) := \binom{d}{k} s^k (1-s)^{d-k}, \quad s \in [0, 1] \quad (2.12)$$

Lưu ý rằng các đa thức Bernstein bậc d luôn không âm và theo định lý nhị thức, tổng của chúng bằng 1. Do đó, với mỗi $s \in [0, 1]$ cố định, các vô hướng $\{\beta_{k,d}(s)\}_{k=0}^d$ có thể được xem như các hệ số của một tổ hợp lồi. Các đường cong Bézier thu được bằng cách sử dụng các hệ số này để tổ hợp một tập hợp gồm $d+1$ điểm điều khiển $\gamma_k \in \mathbb{R}^n$ đã cho:

$$\gamma(s) := \sum_{k=0}^d \beta_{k,d}(s) \gamma_k \quad (2.13)$$

Có thể dễ dàng kiểm chứng rằng các đường cong Bézier sở hữu các tính chất sau đây, vốn đóng vai trò thiết yếu đối với khuôn khổ GCS:

- **Giá trị điểm cuối (Endpoint values):** Đường cong γ bắt đầu tại điểm điều khiển đầu tiên và kết thúc tại điểm điều khiển cuối cùng:

$$\gamma(0) = \gamma_0 \quad \text{và} \quad \gamma(1) = \gamma_d \quad (2.14)$$

Tính chất này rất cần thiết cho việc thực thi các ràng buộc liên tục (C^0) khi ghép nối nhiều đoạn Bézier lại với nhau trong đồ thị.

- **Tính chất bao lồi (Convex hull property):** Đường cong γ nằm hoàn toàn bên trong bao lồi của các điểm điều khiển của nó:

$$\gamma(s) \in \text{conv}(\{\gamma_k\}_{k=0}^d), \quad \forall s \in [0, 1] \quad (2.15)$$

Trong bối cảnh của GCS, nếu một vùng lồi X_v đại diện cho một tập an toàn, việc đảm bảo tránh va chạm được quy về việc ràng buộc tất cả các điểm điều khiển phải nằm trong tập hợp đó:

$$\gamma_k \in X_v, \quad \forall k \in \{0, \dots, d\} \implies \gamma(s) \in X_v \quad (2.16)$$

Điều này cho phép chúng ta thực thi các ràng buộc an toàn liên tục thông qua một tập hợp hữu hạn các ràng buộc bao hàm tuyến tính.

- **Đạo hàm (Hodograph):** Đạo hàm $\dot{\gamma}$ của đường cong γ cũng là một đường cong Bézier, nhưng có bậc là $d-1$, với các điểm điều khiển được xác định bởi sai phân của các điểm kề nhau:

$$\dot{\gamma}(s) = \sum_{k=0}^{d-1} \beta_{k,d-1}(s) \dot{\gamma}_k \quad (2.17)$$

trong đó $\dot{\gamma}_k = d(\gamma_{k+1} - \gamma_k)$ với $k = 0, \dots, d-1$. Tương tự, các đạo hàm bậc cao hơn (gia tốc, độ giật) có thể được biểu diễn dưới dạng tổ hợp tuyến tính của các điểm điều khiển. Do đó, các ràng buộc động lực học (ví dụ: giới hạn vận tốc $|\dot{\gamma}(s)| \leq v_{max}$) có thể được áp đặt dưới

dạng các ràng buộc tuyến tính trên sai phân giữa các điểm điều khiển kế nhau, qua đó bảo toàn tính lồi của bài toán tối ưu hóa.

- **Tích phân của hàm lồi:** Đối với một hàm lồi $f : \mathbb{R}^n \rightarrow \mathbb{R}$ (thường đại diện cho hàm chi phí trong lập kế hoạch chuyển động, chẳng hạn như năng lượng hoặc độ dài đường đi), tích phân dọc theo đường cong bị chặn bởi tổ hợp lồi của các giá trị hàm tại các điểm điều khiển:

$$\int_0^1 f(\gamma(s))ds \leq \frac{1}{d+1} \sum_{k=0}^d f(\gamma_k) \quad (2.18)$$

Bất đẳng thức này cung cấp một cận trên lồi cho chi phí tích phân, tạo điều kiện thuận lợi cho việc tối ưu hóa hiệu quả trong khuôn khổ bài toán quy hoạch lồi hỗn hợp số nguyên (mixed-integer convex programming) của GCS.

2.4 Phân hoạch Lồi

Phần này cung cấp tổng quan về phân hoạch lồi, một kỹ thuật cơ bản được sử dụng trong hình học tính toán và mô phỏng vật lý.

2.4.1 Định nghĩa

Phân hoạch lồi (Convex Decomposition) là quá trình chia một hình học phức tạp (ví dụ: đa giác hoặc đa diện không lồi) thành một tập hợp các **vùng con lồi** sao cho hợp của các vùng này khôi phục lại toàn bộ hình ban đầu và các vùng con không chồng lấn nhau, ngoại trừ biên chung.

Về mặt hình thức, với một miền hình học $P \subset \mathbb{R}^n$, một *phân hoạch lồi* của P là tập hợp $\{P_1, P_2, \dots, P_k\}$ sao cho:

$$P = \bigcup_{i=1}^k P_i, \quad P_i \cap P_j = \partial P_i \cap \partial P_j \text{ với mọi } i \neq j,$$

và mỗi P_i là một tập lồi. Mục tiêu là tìm tập phân hoạch này như là số lượng các vùng lồi tối thiểu, giảm thiểu phần chồng chéo hay chất lượng của vùng lồi được tạo ra (tránh quá "mảnh" hoặc "dài").

2.4.2 Phân loại Phương pháp Phân hoạch Lồi

Trong lĩnh vực hình học tính toán và hoạch định chuyển động, các phương pháp phân hoạch lồi thường được chia thành hai loại chính: **Phân hoạch Lồi Chính xác (Exact Convex Decomposition – ECD)** và **Phân hoạch Lồi Xấp xỉ (Approximate Convex Decomposition – ACD)**. Sự khác biệt giữa hai loại này phản ánh sự đánh đổi cơ bản giữa *tính chính xác hình học* và *hiệu quả tính toán*.

2.4.2.1 Phân hoạch Lồi Chính xác (Exact Convex Decomposition – ECD)

Phân hoạch Lồi Chính xác (ECD) là quá trình chia một hình dạng không lồi thành các **thành phần con hoàn toàn lồi**, không chồng lấn, sao cho hợp của chúng tái tạo lại chính xác hình dạng ban đầu. Mục tiêu thường là tối thiểu hóa số lượng vùng lồi.

Tuy nhiên, ECD gặp hai thách thức lớn:

- **Độ phức tạp tính toán cao:** Bài toán tìm phân hoạch lồi tối thiểu đã được chứng minh là NP-hard đối với đa giác có lỗ, nên không tồn tại thuật toán hiệu quả cho các trường hợp tổng quát.
- **Tính khả dụng hạn chế:** Ngay cả khi tính toán được, ECD thường tạo ra số lượng vùng rất lớn, khiến việc lưu trữ, xử lý và tối ưu hóa tiếp theo trở nên tốn kém.

Do đó, các phương pháp ECD chủ yếu mang tính lý thuyết, dùng làm chuẩn so sánh, trong khi hầu hết các ứng dụng thực tế trong robot và mô phỏng đều dựa vào các phương pháp xấp xỉ.

2.4.2.2 Phân hoạch Lỗi Xấp xỉ (Approximate Convex Decomposition – ACD)

Trước những giới hạn của ECD, các phương pháp **Phân hoạch Lỗi Xấp xỉ (ACD)** được phát triển để đạt sự cân bằng giữa độ chính xác và hiệu quả. ACD cho phép các thành phần không hoàn toàn lỗi, mà chỉ “*gần lỗi*” trong một mức dung sai lờm τ do người dùng xác định hoặc chỉ bao phủ một phần không gian tự do. Triết lý cốt lõi của ACD là chấp nhận một **mức độ lờm nhỏ có kiểm soát để đổi lấy**:

- **Hiệu quả tính toán cao hơn:** Các thuật toán ACD, chẳng hạn như phương pháp của Lien và Amato, chạy nhanh hơn nhiều so với ECD. [8]
- **Giảm mạnh số lượng vùng:** Cho phép biểu diễn hình dạng với ít vùng lỗi hơn, đơn giản hơn và dễ xử lý hơn.
- **Kiểm soát mức độ chi tiết:** Một số thuật toán ACD như phương pháp của Lien và Amato cho phép người dùng điều chỉnh tham số lờm τ để cân bằng giữa độ chính xác và số lượng vùng lỗi. Hay đối với phương pháp Visibility Clique Cover (VCC), nó cho phép người dùng chọn chỉ số để thể hiện mức độ bao phủ của vùng lỗi so với không gian tự do ban đầu. [9]

Chapter 3

Related works

...

3.1 Sampling-Based Methods

3.2 Discussion

In this chapter, we have seen three distinct approaches to enhance...

Chapter 4

Proposed Solution

...

4.1 Phân hoạch không gian an toàn

Thay vì tiếp cận bài toán tránh vật cản theo cách truyền thống là mô tả các ràng buộc không lỗi dựa trên bề mặt vật cản, nhóm tác giả đề xuất một phương pháp chuyển đổi không gian cấu hình tự do (collision-free configuration space) thành hợp của một tập hợp hữu hạn các vùng lỗi bị chặn (bounded convex regions). Trong mô hình này, các vùng lỗi có thể chồng lấn lên nhau, và bài toán hoạch định chuyển động được phát biểu lại dưới dạng một chương trình quy hoạch lồi nguyên hỗn hợp (Mixed-Integer Convex Program - MICP). Cụ thể, các biến nguyên được sử dụng để gán từng đoạn quỹ đạo đa thức (polynomial trajectory segments) vào các vùng lỗi an toàn cụ thể này. Phương pháp này mang lại lợi thế lớn về mặt tính toán vì số lượng biến nguyên chỉ phụ thuộc vào số lượng vùng an toàn được tạo ra (sử dụng thuật toán IRIS) thay vì phụ thuộc vào số lượng mặt của vật cản, giúp giải quyết hiệu quả các môi trường phức tạp với hàng trăm vật cản. Hơn nữa, việc ràng buộc quỹ đạo nằm trọn trong các tập lỗi đảm bảo an toàn tuyệt đối cho toàn bộ hành trình liên tục, khắc phục nhược điểm "cắt góc" (corner cutting) thường gặp ở các phương pháp chỉ kiểm tra

va chạm tại các điểm lấy mẫu rời rạc.

4.2 Các Phương pháp Cốt lõi cho Phân hoạch Không gian Tự do

Để giải quyết thách thức về tính toán của phân hoạch tối ưu, một số các phương pháp đã được phát triển. Dưới đây là một số phương pháp được dùng trong bài báo cáo này:

4.2.1 Phân hoạch Lỗi Xấp xỉ (ACD) của Đa giác

Phương pháp Approximate Convex Decomposition (ACD) giới thiệu một cách tiếp cận khác biệt cơ bản để quản lý sự phức tạp. Thay vì yêu cầu sự lồi hoàn hảo, nó cho phép các thành phần "lồi xấp xỉ" trong một dung sai do người dùng xác định, τ . [8] Điều này thường dẫn đến các phân hoạch có ít thành phần hơn nhiều và phù hợp hơn với các đặc điểm cấu trúc nổi bật của đối tượng.

4.2.1.1 Các Khái niệm Cơ bản trong Phân rã Lồi

Phần này trình bày các khái niệm hình học cơ bản được sử dụng trong quá trình phân rã đa giác không lồi thành các vùng lồi. Các định nghĩa này đóng vai trò nền tảng cho các phương pháp đo độ lồi và các thuật toán phân hoạch ở các phần sau.

- **Bao lồi (Convex Hull – H_P):** Bao lồi của một đa giác P là **tập hợp lồi nhỏ nhất chứa toàn bộ đa giác** đó. Trực quan, có thể hình dung bao lồi như một sợi dây chun được kéo căng bao quanh đa giác. Một đa giác P được gọi là **lồi** nếu và chỉ nếu nó trùng với bao lồi của chính nó, tức là:

$$P = H_P.$$

- **Notches:** Là các đỉnh của đa giác P **không nằm trên** bao lồi H_P . Về mặt hình học, đây là những đỉnh có **góc trong lớn hơn** 180° . Các Notches biểu hiện các đặc trưng lõm (concave features) trên biên của đa giác.
- **Cầu (Bridges):** Là các cạnh của bao lồi ∂H_P nối hai đỉnh không liên kề của biên ngoài ∂P_0 . Về mặt toán học, tập hợp các cầu được định nghĩa:

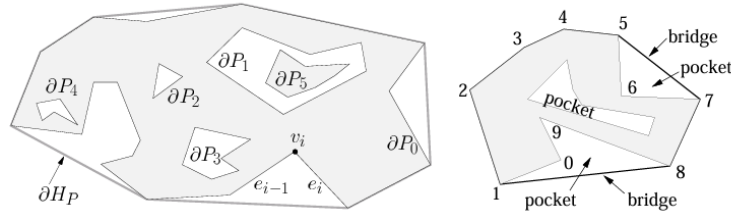
$$\text{BRIDGES}(P) = \partial H_P \setminus \partial P.$$

Mỗi cầu “bắc qua” một vùng lõm của đa giác.

- **Túi (Pockets):** Là các **chuỗi cạnh liên tiếp** của biên đa giác ∂P không thuộc về vỏ lồi ∂H_P . Về mặt toán học:

$$\text{POCKETS}(P) = \partial P \setminus \partial H_P.$$

Mỗi túi tương ứng với một “vịnh lõm” (concave bay) trên đường biên của đa giác.



4.2.1.2 Ý tưởng và Chi tiết Thuật toán

Thuật toán là một chiến lược chia để trị đệ quy[8]:

1. Đo độ lõm: Xác định đặc điểm không lồi (một "notch" hay đỉnh lõm) quan trọng nhất trong đa giác. Đây là điểm có độ lõm lớn nhất. Tùy theo từng trường hợp mà ta có thể sử dụng các thước đo độ lõm khác

nhau, cách để tính những thước đo độ lõm này sẽ được giới thiệu rõ hơn ở phần 4.2.1.3

2. Kiểm tra Dung sai: Nếu độ lõm tối đa đo được nhỏ hơn τ , quá trình kết thúc đối với thành phần đó.
3. Giải quyết Đỉnh lõm: Nếu độ lõm vượt quá τ , một đường cắt được thêm vào để giải quyết đỉnh lõm, chia đa giác thành hai thành phần mới (hoặc hợp nhất một lỗ).

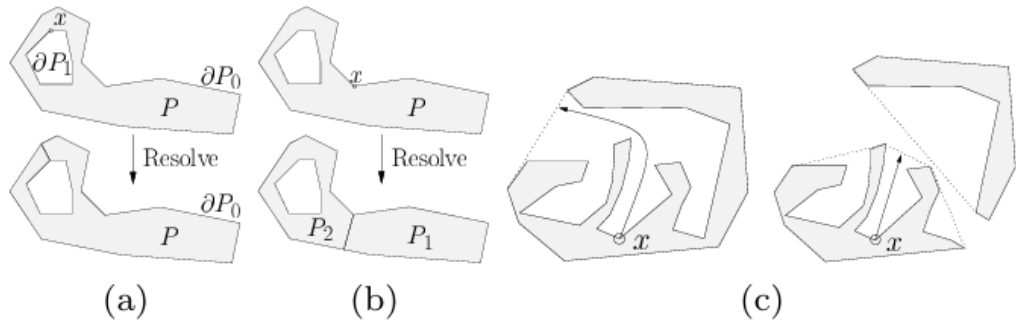


Figure 4.1: (a) Nếu $x \in \partial P_i > 0$, hàm **Resolve** **gộp** biên ∂P_i vào đa giác P_0 . (b) Nếu $x \in \partial P_0$, hàm **Resolve** **tách** đa giác P thành hai đa giác P_1 và P_2 . (c) **Độ lõm** tại điểm x thay đổi sau khi đa giác được phân rã.

4. Độ quy: Quá trình được áp dụng đệ quy cho các thành phần mới.

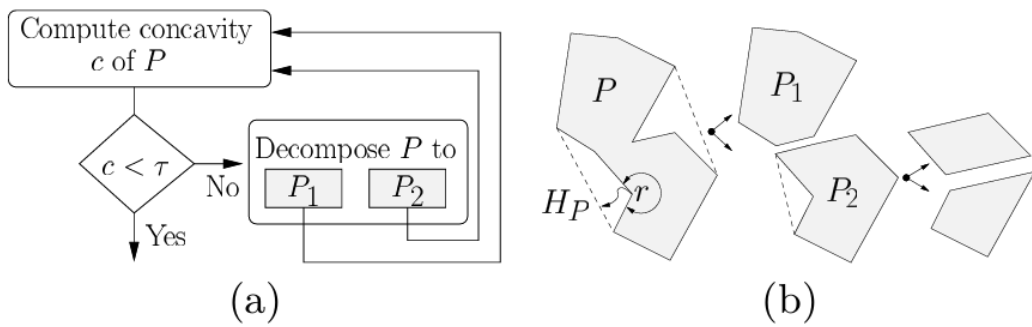


Figure 4.2: Quá trình đệ quy tiếp tục cho đến khi đạt đến giới hạn độ lõm đầu vào của người dùng τ .

4.2.1.3 Các Thước đo Độ lõm

Chất lượng của phân hoạch phụ thuộc rất nhiều vào **cách đo độ lõm** của các đỉnh trong đa giác không lồi. Các phương pháp đo độ lõm phổ biến bao gồm:

- **Độ lõm Đường thẳng (Straight-Line Concavity – SL-Concavity):**

Là khoảng cách Euclid từ một đỉnh trong *túi* (pocket) đến *cầu nối* (bridge) liên kết của nó — thường là một cạnh thuộc bao lồi của đa giác. Phương pháp này rất nhanh, nhưng **không đảm bảo tính đơn điệu**, có thể khiến một số đặc trưng lõm quan trọng bị bỏ sót.



Figure 4.3: Giả sử r là điểm lõm có độ lõm lớn nhất. Sau khi giải quyết (resolve) r , độ lõm của s tăng lên. Nếu $\text{concave}(r) < \tau$, vùng s sẽ không bao giờ được xử lý, ngay cả khi $\text{concave}(s)$ lớn hơn τ .

- **Độ lõm Đường đi Ngắn nhất (Shortest Path Concavity – SP-Concavity):** Là **chiều dài đường đi ngắn nhất** từ một đỉnh trong túi đến cầu nối tương ứng, với điều kiện đường đi đó phải nằm hoàn toàn trong túi. Phương pháp này chậm hơn SL-Concavity nhưng **đảm bảo tính đơn điệu**: độ lõm của các đỉnh giảm dần sau mỗi lần phân rã, giúp đảm bảo rằng tất cả các đặc điểm lõm quan trọng cuối cùng sẽ được xử lý. (Xem chi tiết trong Phần ??.)
- **Độ lõm Lai (Hybrid Concavity – H-Concavity):** Một giải pháp trung gian thực tiễn — sử dụng SL-Concavity nhanh làm mặc định và chỉ chuyển sang SP-Concavity khi **phát hiện khả năng không đơn điệu**. Bằng cách thêm bước kiểm tra *nguy cơ tiềm ẩn* bằng cách so sánh n_β và n_i ; nếu tồn tại $n_i \cdot n_\beta < 0$, biên túi bị đảo hướng — dấu

hiệu của túi phức tạp mà SL-Concavity có thể thất bại.

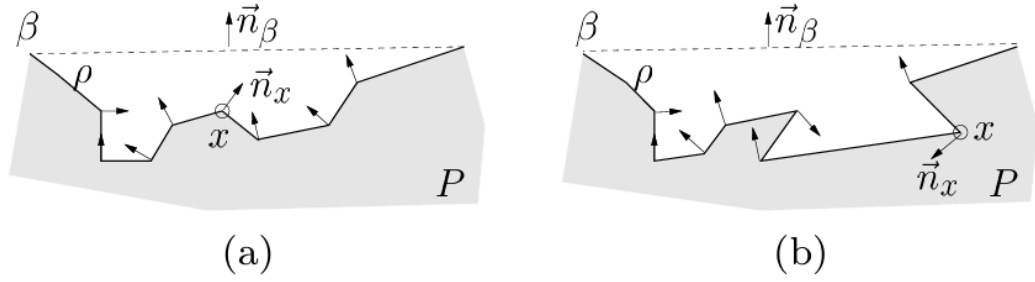


Figure 4.4: Độ lõm SL có thể xử lý được túi trong (a) một cách chính xác vì không có hướng chuẩn nào của các đỉnh trong túi ngược với hướng chuẩn của cầu. Tuy nhiên, túi trong (b) có thể dẫn đến độ lõm giảm không đơn điệu.

4.2.1.4 Đặc tính của Thuật toán

- Loại thuật toán: Đây là một phương pháp xấp xỉ theo định nghĩa. Một phân hoạch với $\tau = 0$ là một phân hoạch lỗi chính xác.
- Độ phức tạp thời gian: $O(nr^2)$, trong đó n là số đỉnh và r là số đỉnh lõm.[8] Điều này nhanh hơn các thuật toán phân hoạch chính xác tối ưu cho đa giác không có lỗ.

4.2.2 Iterative Regional Inflation by Semi-Definite Programming (IRIS)

Thuật toán Iterative Regional Inflation by Semidefinite Programming (IRIS) thay vì cố gắng bao phủ toàn bộ không gian tự do cùng một lúc, nó đặt ra câu hỏi: "Với một điểm 'mầm' (seed) ban đầu, vùng lõi lớn nhất của không gian tự do mà tôi có thể tìm thấy xung quanh nó là gì?". Tức sau khi chạy xong giải thuật output của ta là một vùng lõi lớn nhất có thể của thuật toán từ 1 seed point cho trước.

4.2.2.1 Ý tưởng và Chi tiết Thuật toán

IRIS là một quy trình tối ưu hóa lặp đi lặp lại gồm 4 bước:

1. **Khởi tạo:** Thuật toán sẽ tạo ra một hình elip ban đầu là một hình cầu rất nhỏ có tâm chính là điểm mầm (seed point).
2. **Tìm Siêu phẳng Phân cách (QP):**

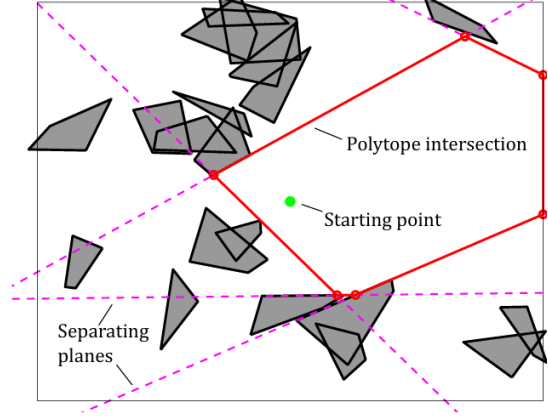


Figure 4.5: Siêu phẳng Phân cách

Đối với ellipsoid hiện tại, thuật toán tìm điểm gần nhất trên mỗi chướng ngại vật. Sau đó, nó xây dựng các siêu phẳng phân cách ellipsoid khỏi các chướng ngại vật này. Mỗi siêu phẳng tiếp tuyến với một phiên bản mở rộng của ellipsoid và đi qua điểm gần nhất trên chướng ngại vật tương ứng. Việc tìm điểm gần nhất trên mỗi chướng ngại vật lỗi được xây dựng như một bài toán Quy hoạch Toàn phương (Quadratic Program - QP). Để giữ cho bài toán tối ưu hóa tiếp theo có quy mô nhỏ, các siêu phẳng thừa (những siêu phẳng không xác định biên của vùng lỗi kết quả) sẽ được loại bỏ.

Bước 2.1. Tìm điểm gần nhất trên obstacle đến hình elip hiện tại

- **Biến đổi không gian (Từ không gian Ellipsoid sang không gian Quả cầu đơn vị)**

Việc tính toán khoảng cách đến một ellipsoid là phức tạp. Tuy nhiên, tính khoảng cách đến một quả cầu đơn vị tại gốc tọa độ thì lại rất đơn giản. Ý tưởng ở đây là biến đổi toàn bộ không gian sao cho ellipsoid trở thành một quả cầu đơn vị.

- Một ellipsoid E được định nghĩa là ảnh của một quả cầu đơn vị $\tilde{E}$ qua phép biến đổi affine:

$$x = C\tilde{x} + d.$$

- Sử dụng phép biến đổi ngược:

$$\tilde{x} = C^{-1}(x - d),$$

để ánh xạ mọi điểm trở lại “không gian quả cầu đơn vị”.

- Trong không gian mới này, ellipsoid trở thành quả cầu đơn vị $\tilde{E}$ tại gốc tọa độ, và mỗi chương ngại vật l_j cũng bị biến đổi (méo đi) thành một đối tượng mới $\tilde{l}_j$.

Bước 2.2: Tìm điểm gần nhất bằng Quy hoạch Toàn phương (QP)

Bây giờ, bài toán tìm điểm trên chương ngại vật l_j gần ellipsoid E nhất tương đương với việc tìm điểm trên chương ngại vật đã biến đổi $\tilde{l}_j$ gần gốc tọa độ nhất.

Bài toán này được xây dựng dưới dạng một bài toán *Quy hoạch Toàn phương* (Quadratic Program - QP):

$$\begin{aligned} & \underset{\tilde{x} \in \mathbb{R}^n, w \in \mathbb{R}^m}{\text{minimize}} && \|\tilde{x}\|^2 \\ & \text{subject to} && \begin{cases} [\tilde{v}_{j,1} \ \tilde{v}_{j,2} \ \cdots \ \tilde{v}_{j,m}] w = \tilde{x}, \\ \sum_{i=1}^m w_i = 1, \\ w_i \geq 0, \quad i = 1, \dots, m. \end{cases} \end{aligned}$$

Về bản chất, ta đang tìm một điểm $\tilde{x}$ là **tổ hợp lồi** của các đỉnh đã biến đổi $\tilde{v}_{j,k}$ sao cho khoảng cách của $\tilde{x}$ đến gốc tọa độ là nhỏ nhất.

Bước 2.3. Xác định mặt phẳng tiếp tuyến của ellipsoid tại điểm tiếp xúc

Sau khi tìm được điểm x^* gần nhất, ta xây dựng mặt phẳng tiếp tuyến với ellipsoid tại điểm đó. Ellipsoid được mô tả bằng biểu thức nghịch đảo:

$$E = \{x \mid (x - d)^\top C^{-1} C^{-\top} (x - d) \leq 1\}.$$

Vector pháp tuyến của ellipsoid tại x^* được xác định bởi gradient:

$$a_j = \nabla_x \left[(x - d)^\top C^{-1} C^{-\top} (x - d) \right]_{x=x^*} = 2C^{-1} C^{-\top} (x^* - d).$$

Vì mặt phẳng tiếp tuyến đi qua x^* , ta có:

$$b_j = a_j^\top x^*.$$

Khi đó, phương trình mặt phẳng tiếp tuyến là:

$$a_j^\top x = b_j,$$

được thêm vào như một **ràng buộc tuyến tính** trong tập khả thi của thuật toán IRIS. Mặt phẳng này đảm bảo ellipsoid mở rộng tối đa nhưng không giao với chướng ngại vật.

Bước 2.4. Lặp lại với tất cả các obstacles trong không gian. Ta sẽ có được một tập các siêu phẳng để tạo thành một vùng lồi đa giác.

3. **Tìm Ellipsoid Nội tiếp Cực đại (SDP):** Giao của các nửa không gian được xác định bởi các siêu phẳng tạo thành một đa diện lồi (polytope). Thuật toán sau đó tìm ellipsoid có thể tích lớn nhất có thể nội tiếp trong đa diện này.

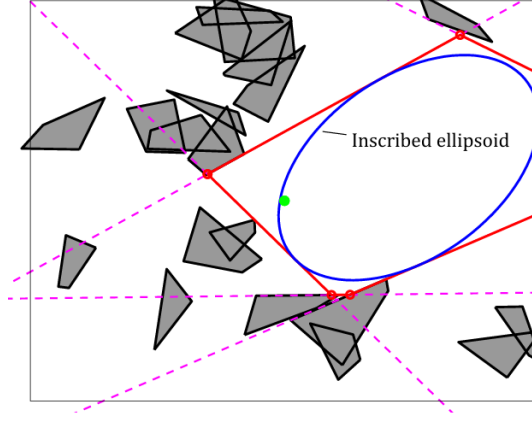


Figure 4.6: Ellipsoid Nội tiếp Cực đại

Bài toán này tìm **ellipsoid có thể tích lớn nhất** nằm gọn trong một đa diện lồi P :

$$P = \{x \in \mathbb{R}^n \mid Ax \leq b\}. \quad (4.1)$$

Ellipsoid được định nghĩa là $\mathcal{E}$, với d là tâm và ma trận C xác định hình dạng:

$$\mathcal{E} = \{C\tilde{x} + d \mid \|\tilde{x}\|_2 \leq 1\}. \quad (4.2)$$

Thể tích của ellipsoid tỉ lệ với $\det(C)$.

Để tìm ellipsoid lớn nhất nội tiếp trong P , ta giải bài toán tối ưu lồi sau:

$$\begin{aligned} & \underset{C, d}{\text{maximize}} && \log \det C \\ & \text{subject to} && \|a_i^\top C\|_2 + a_i^\top d \leq b_i, \quad \forall i = 1, \dots, N, \\ & && C \succ 0. \end{aligned} \quad (4.3)$$

Mục tiêu: Cực đại hoá $\log \det C$ để tối đa hoá thể tích ellipsoid.

Ràng buộc: Đảm bảo mọi điểm của ellipsoid đều nằm trong đa diện

P .

4. **Lặp lại:** Ellipsoid mới, lớn hơn này được sử dụng làm điểm khởi đầu cho vòng lặp tiếp theo. Quá trình này hội tụ khi thể tích của ellipsoid ngừng tăng một cách đáng kể, trả về một đa diện lồi lớn và ellipsoid nội tiếp của nó.

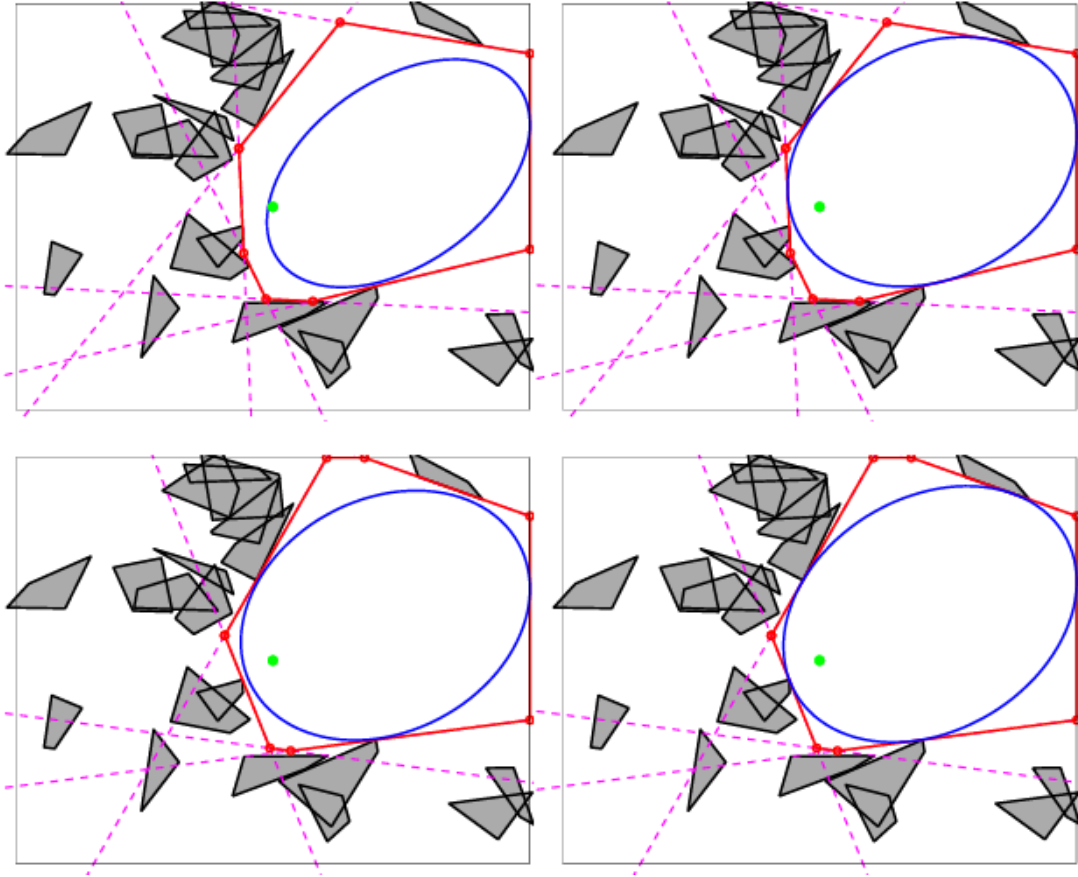


Figure 4.7: Lặp lại

4.2.2.2 Đặc tính của Thuật toán

- Loại thuật toán: IRIS là một phương pháp xấp xỉ để bao phủ toàn bộ không gian tự do. Một lần chạy duy nhất chỉ tạo ra một vùng lõi. Để có được một lớp phủ hoàn chỉnh, cần phải chạy thuật toán nhiều lần với các điểm mầm khác nhau.
- Độ phức tạp thời gian: IRIS thường hội tụ qua 4-8 vòng lặp. Thời gian tính toán của mỗi vòng lặp bằng thời gian tìm các mặt phẳng phân cách cộng với thời gian tính toán ellipses nội tiếp. Trong khi thời gian giải ellipses (SDP) gần như không đổi (constant) nhờ vào việc loại bỏ các siêu phẳng thừa thì thời gian tính các siêu phẳng phân cách lại tăng tuyến tính theo số lượng các obstacles.

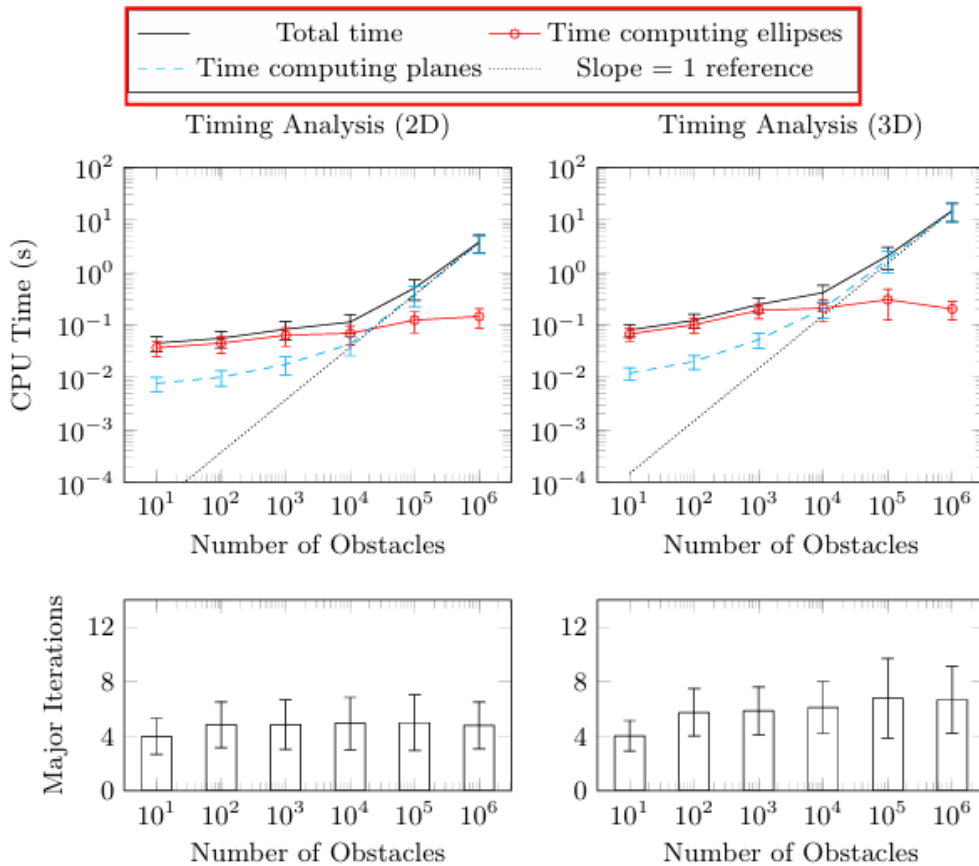


Figure 4.8: Độ phức tạp thời gian [10]

- Xử lý Vật cản không lõi:

- Chướng ngại vật trong không gian work space: Thuật toán IRIS ban đầu giả định các chướng ngại vật là lỗi, đây cũng là một nhược điểm lớn nhất của thuật toán này. Để đảm bảo các obstacles là lỗi, ta thường phải tiền xử lý bằng cách bao lỗi các chướng ngại vật không lỗi[10]. Ngoài ra cách tiếp cận tiêu chuẩn cho các chướng ngại vật không lỗi là phân hoạch chúng thành một tập hợp các mảnh lỗi trước khi chạy IRIS (tức tăng số lượng obstacles của không gian lên). Điều này khả thi vì IRIS có khả năng mở rộng hiệu quả với số lượng lớn chướng ngại vật.
- Chướng ngại vật trong không gian cấu hình (Configuration Space): Đối với các robot có động học phức tạp, nơi các chướng ngại vật trong không gian cấu hình được định nghĩa một cách ẩn ý và không lỗi, các phần mở rộng như IRIS-NP và C-IRIS được sử dụng. Đặc biệt, bài báo "Motion Planning around Obstacles with Convex Optimization" sử dụng một triển khai có tên là Iris-InConfigurationSpace trong thư viện Drake cho ví dụ về cánh tay robot. **motionplanning2022** Phần mở rộng này sử dụng lập trình phi tuyến (nonlinear programming) để xử lý hình học phức tạp của không gian cấu hình.

4.2.3 Gieo mầm Tự động qua Lớp phủ Clique Khả kiến (VCC)

Thuật toán Visibility Clique Cover (VCC) giải quyết trực tiếp một hạn chế chính của IRIS: nhu cầu về các điểm mầm tốt.⁵ Việc lấy mầm ngẫu nhiên là không hiệu quả. VCC tự động hóa quá trình này bằng cách trước tiên tìm hiểu cấu trúc toàn cục của không gian tự do và sau đó đặt các điểm mầm một cách chiến lược vào các khu vực có khả năng mở rộng lớn.

4.2.3.1 Ý tưởng và Chi tiết Thuật toán

Quy trình VCC bao gồm bốn bước chính:

1. Lấy mẫu & Xây dựng Đồ thị Khả kiến (Visibility Graphs): Thuật toán lấy mẫu n điểm ngẫu nhiên trong không gian cấu hình tự do (C_{free}). Các điểm này tạo thành các đỉnh của một đồ thị khả kiến (visibility graph)¹. Một cạnh tồn tại giữa hai đỉnh nếu đoạn thẳng nối hai điểm tương ứng hoàn toàn nằm trong không gian tự do (tức là không va chạm).

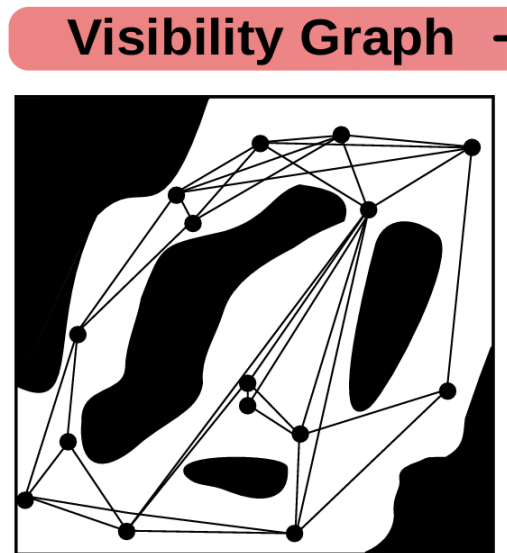


Figure 4.9: Lấy mẫu & Xây dựng Đồ thị Khả kiến (Visibility Graphs)

2. Tìm Lớp phủ Clique: Thuật toán tìm một tập hợp các clique lớn (đồ thị con đầy đủ) trong đồ thị khả kiến. Ý tưởng cốt lõi là một tập hợp các điểm mà tất cả đều "nhìn thấy" nhau có khả năng nằm trong cùng một vùng lồi². Điều này được thực hiện một cách tham lam bằng cách giải lặp đi lặp lại bài toán MAXCLIQUE (tìm clique lớn nhất), được xây dựng dưới dạng một bài toán Integer Linear Programming - ILP.

¹ *Visibility graph* là đồ thị trong đó các đỉnh biểu diễn vị trí (hoặc đỉnh chướng ngại vật), và có cạnh nối giữa hai đỉnh nếu đoạn thẳng nối chúng nằm hoàn toàn trong không gian tự do.

² Nếu hai điểm có thể nhìn thấy nhau, điều đó ngụ ý rằng con đường thẳng nhất giữa chúng là an toàn. Đây là khối xây dựng cơ bản để hiểu và xấp xỉ các vùng lồi trong.

► Clique Cover —

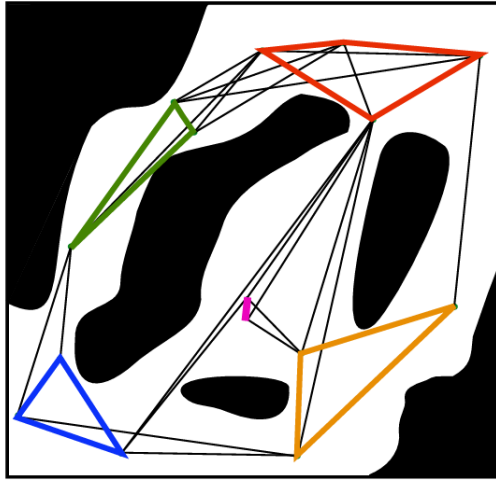


Figure 4.10: Tìm Lớp phủ Clique

3. Khởi tạo Ellipsoid: Đối với mỗi clique được tìm thấy, thuật toán tính toán ellipsoid có thể tích nhỏ nhất bao quanh tất cả các điểm trong clique đó. Ellipsoid này cung cấp một tâm (điểm mầm) và các trục chính (hình dạng ban đầu) cho bước lấp đầy tiếp theo.

► Ellipsoids —

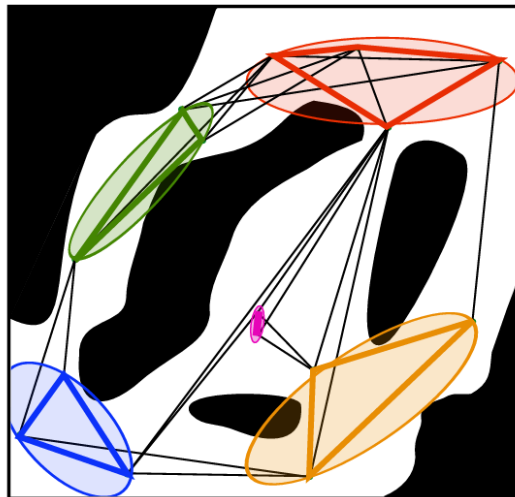


Figure 4.11: Khởi tạo Ellipsoid

4. Lấp đầy thành Đa diện: Tâm và hình dạng của mỗi ellipsoid được sử dụng để khởi tạo một vòng lặp duy nhất của thuật toán lấp đầy tương

tự IRIS. Điều này hiệu quả hơn IRIS tiêu chuẩn vì ellipsoid ban đầu đã được "thông báo" về hình học cục bộ, thường loại bỏ sự cần thiết của nhiều vòng lặp tốn kém.⁵

► Inflate Polytopes -

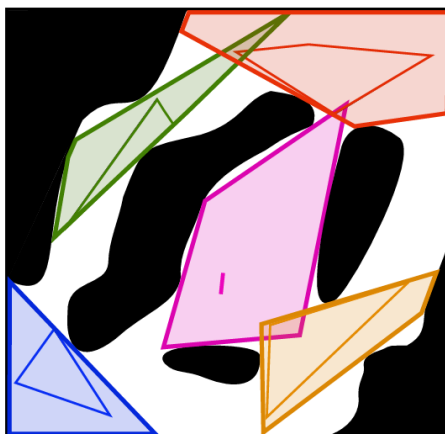


Figure 4.12: Lấp đầy thành Đa diện

5. Lặp lại và Hội tụ: Quá trình này được lặp lại cho đến khi đạt được độ phủ đủ bằng cách lấy mẫu mới từ không gian trống còn lại và lặp lại các bước trước đó.

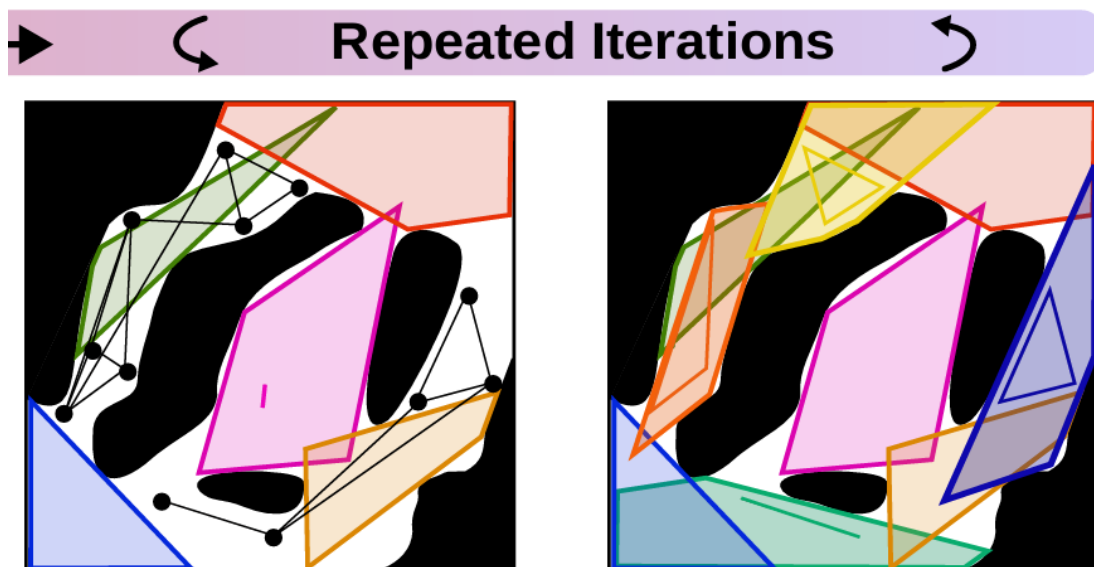


Figure 4.13: Lặp lại

4.2.3.2 Đặc tính và Tính tối ưu

- Loại thuật toán: VCC là một phương pháp xấp xỉ. Chất lượng của lớp phủ được đo bằng tỷ lệ phần trăm thể tích của C_{free} được bao phủ bởi hợp của các đa diện kết quả.⁵
- Mục tiêu Tối ưu: Thuật toán nhằm tìm một lớp phủ lồi xấp xỉ α (α -approximate convex cover) với số lượng vùng tối thiểu. Nó sử dụng bài toán lớp phủ clique tối thiểu (minimum clique cover) như một bài toán tương tự rời rạc cho bài toán lớp phủ lồi tối thiểu.[9]
- Phương pháp: Đây là một cách tiếp cận lai ghép giữa hình học và ILP.
- Xử lý Không gian không lồi: VCC không phụ thuộc vào hình dạng của chướng ngại vật vì nó dựa vào một bộ kiểm tra va chạm chung để kiểm tra khả kiến và sau đó sử dụng một thuật toán lấp đầy cơ bản (như IRIS-NP) có thể xử lý các không gian cấu hình không lồi.[9]

4.3 Đồ thị các tập lồi

Khung phương pháp *Graph of Convex Sets* (GCS) cung cấp một cách tiếp cận thống nhất để giải các bài toán tối ưu lai rời rạc–liên tục, đóng vai trò cầu nối giữa tìm kiếm tổ hợp trên đồ thị và lập trình lồi liên tục. Khác với lý thuyết đồ thị cổ điển, GCS mở rộng bài toán Đường đi ngắn nhất (Shortest Path Problem – SPP) bằng cách gắn trực tiếp các biến hình học liên tục vào cấu trúc đồ thị.

4.3.1 Mô hình bài toán Đường đi ngắn nhất trong Graph of Convex Sets

Một GCS được định nghĩa là một đồ thị có hướng $G = (V, E)$, trong đó mỗi đỉnh $v \in V$ được gắn với một biến quyết định liên tục x_v , bị ràng buộc

trong một tập lồi, không rỗng và compact $X_v \subset \mathbb{R}^n$. Trong bối cảnh lập kế hoạch chuyển động, các tập này thường biểu diễn các vùng không va chạm đã được tiền xử lý trong không gian cấu hình. Các chuyển tiếp giữa các đỉnh được mô tả bởi các cạnh $e = (u, v) \in E$, mỗi cạnh áp đặt một tập ràng buộc lồi phụ $(x_u, x_v) \in X_e$ và được gán một hàm chi phí lồi $\ell_e(x_u, x_v)$. Hàm chi phí này được yêu cầu là proper, closed và lồi, nhằm liên kết trực tiếp các biến liên tục x_u và x_v để mô hình hóa các đại lượng chuyển tiếp như khoảng cách Euclid hoặc năng lượng tiêu thụ. Cần lưu ý rằng, mặc dù được gọi là “chi phí”, ℓ_e không nhất thiết phải là một metric hợp lệ; các tính chất như đối xứng hay bất đẳng thức tam giác không được giả định là thỏa mãn **gcs2023**.

Trong khi mô hình luồng mạng cổ điển giải hiệu quả bài toán SPP trên các đồ thị rời rạc với chi phí cạnh cố định, khung GCS tổng quát hóa cách tiếp cận này bằng cách đưa vào các biến quyết định liên tục gắn với cấu trúc đồ thị. Trong GCS, chi phí khi đi qua một cạnh $e = (u, v)$ không còn là một hằng số c_{uv} , mà là một hàm lồi $\ell_e(x_u, x_v)$ phụ thuộc vào trạng thái liên tục $x_u \in X_u$ và $x_v \in X_v$ tại các đỉnh kề. Mục tiêu của bài toán SPP trong GCS là xác định một đường đi p từ đỉnh nguồn s đến đỉnh đích t sao cho tổng chi phí các cạnh trên đường đi là nhỏ nhất, đồng thời lựa chọn các biến liên tục phù hợp tại mỗi đỉnh. Bài toán này được phát biểu một cách hình thức như sau:

$$\min_{p, \{x_v\}_{v \in p}} \sum_{(u,v) \in E_p} \ell_{(u,v)}(x_u, x_v) \quad (4.4)$$

$$\text{s.t. } p \in \mathcal{P}, \quad (4.5)$$

$$x_v \in X_v, \quad \forall v \in p, \quad (4.6)$$

$$(x_u, x_v) \in X_e, \quad \forall e = (u, v) \in E_p \quad (4.7)$$

trong đó $\mathcal{P}$ ký hiệu tập tất cả các đường đi khả thi từ s đến t , và E_p là tập

các cạnh thuộc đường đi p .

4.3.2 Phân tích độ phức tạp

Độ phức tạp tính toán của bài toán SPP trong GCS khác biệt một cách căn bản so với bài toán Đường đi ngắn nhất trong đồ thị rời rạc cổ điển. Trong lý thuyết đồ thị cổ điển, bài toán SPP nói chung là NP-hard (ví dụ như bài toán Đường đi dài nhất hoặc các biến thể có chu trình âm). Tuy nhiên, bài toán này có thể được giải trong thời gian đa thức nếu thỏa mãn hai điều kiện then chốt:

1. Đồ thị là **không chu trình** (cho phép giải bằng sắp xếp topo).
2. Mọi chi phí đỉnh và cạnh đều **không âm** (giải được bằng thuật toán Dijkstra).

Proposition 1. *Trái ngược hoàn toàn với trường hợp trên, bài toán Đường đi ngắn nhất trong Graph of Convex Sets vẫn là **NP-hard** ngay cả khi đồng thời thỏa mãn hai điều kiện: đồ thị không chu trình và chi phí không âm.*

Tính NP-hard này được chứng minh thông qua phép quy giảm từ bài toán **3SAT** (bài toán thỏa mãn Boolean với mỗi mệnh đề gồm ba biến), vốn là một bài toán NP-complete. Như đã trình bày trong Định lý 9.1 của **Marcucci24a**, có thể xây dựng một bài toán GCS dạng phân lớp, không chu trình và có chi phí không âm, sao cho việc xác định sự tồn tại của một đường đi hợp lệ tương đương với việc tìm một gán giá trị thỏa mãn cho công thức 3SAT. Điều này cho thấy rằng ngay cả việc xác định *tính khả thi* của bài toán SPP trong GCS cũng đã là NP-hard.

Các chứng minh khác về tính NP-hard cũng đã được đưa ra cho các trường hợp mà các tập lồi có chiều thấp (1D và 2D), tuy nhiên các trường hợp này thường dựa trên các đồ thị có chu trình **Marcucci24a**.

4.3.3 Mô hình chi tiết các đỉnh và cạnh

Trong bối cảnh tối ưu quỹ đạo, các thành phần của GCS được định nghĩa một cách chặt chẽ nhằm đảm bảo cả tính an toàn hình học lẫn tính khả thi về động học.

Mỗi đỉnh $v \in V$ tương ứng với một vùng lỗi, không va chạm Q_v trong không gian cấu hình (C-space). Biến quyết định liên tục $x_v \in \mathbb{R}^{n_v}$ gắn với đỉnh v biểu diễn một đoạn quỹ đạo cục bộ hoặc một cấu hình trạng thái. Cần nhấn mạnh rằng chiều n_v là cục bộ theo từng đỉnh, cho phép đồ thị có các đỉnh với số chiều biến khác nhau.

Biến x_v bị ràng buộc trong một tập lỗi compact $\mathcal{X}_v \subset \mathbb{R}^{n_v}$. Đối với bài toán sinh quỹ đạo sử dụng các tham số đa thức từng đoạn (ví dụ như đường cong Bézier), tập $\mathcal{X}_v$ được xây dựng một cách tường minh nhằm đảm bảo quỹ đạo nằm hoàn toàn trong vùng an toàn Q_v và thỏa mãn các giới hạn động lực học. Các ràng buộc xác định $\mathcal{X}_v$ thường có dạng:

$$r_{i,k} \in Q_i, \quad k = 0, \dots, d, \quad (4.8)$$

$$\dot{h}_{i,k} \geq \dot{h}_{\min} = 10^{-6}, \quad k = 0, \dots, d-1, \quad (4.9)$$

$$\dot{r}_{i,k} \in \dot{h}_{i,k} D, \quad k = 0, \dots, d-1, \quad (4.10)$$

$$h_{i,0} \geq 0, \quad h_{i,d} \leq T_{\max}. \quad (4.11)$$

Trong đó, (4.8) đảm bảo các điểm điều khiển không gian $r_{i,k}$ nằm trong vùng an toàn. Phương trình (4.9) đảm bảo tính đơn điệu chặt của hàm co giãn thời gian nhằm duy trì tính nhân quả. Phương trình (4.10) liên kết đạo hàm không gian và thời gian để thỏa mãn giới hạn vận tốc được xác định bởi tập D , và (4.11) áp đặt các giới hạn lên thời gian thực hiện quỹ đạo.

Một cạnh $e = (u, v) \in E$ xác định sự chuyển tiếp giữa hai đỉnh. Chuyển tiếp này được điều khiển bởi một *tập ràng buộc liên kết* $\mathcal{X}_e \subseteq \mathbb{R}^{n_u+n_v}$. Tập này

áp đặt các ràng buộc lên vector ghép (x_u, x_v) , đảm bảo các cặp cấu hình (x_u, x_v) là tương thích với nhau, chẳng hạn như thỏa mãn tính liên tục C^0 , C^1 hoặc bậc cao hơn giữa các đoạn quỹ đạo.

Gắn với mỗi cạnh là một *hàm chi phí liên kết* $f_e : \mathbb{R}^{n_u+n_v} \rightarrow \mathbb{R}$. Đây là một hàm lỗi giá trị vô hướng, đánh giá chi phí dựa trên việc lựa chọn đồng thời x_u và x_v , nhằm mô hình hóa các đại lượng như độ dài đường đi hoặc năng lượng tiêu thụ.

4.3.4 Đặc tả hàm chi phí

Việc lựa chọn hàm chi phí cạnh $\ell_e(x_u, x_v)$ quyết định ý nghĩa vật lý của đường đi tối ưu. Trong nghiên cứu này, chúng tôi xem xét hai lớp hàm chi phí chính: chi phí hình học và chi phí quỹ đạo động học.

Đối với các bài toán thuần hình học, chẳng hạn như tìm đường đi Euclid ngắn nhất qua các vùng, chi phí thường được định nghĩa bằng chuẩn L_2 hoặc bình phương của nó. Khoảng cách Euclid bình phương,

$$\ell_e(x_u, x_v) = \|x_v - x_u\|_2^2,$$

là một hàm bậc hai, phù hợp tự nhiên với các bộ giải Quadratic Programming (QP). Ngoài ra, chuẩn L_2 tiêu chuẩn

$$\ell_e(x_u, x_v) = \|x_v - x_u\|_2$$

có thể được mô hình hóa bằng Second-Order Cone Programming (SOCP) trong khi vẫn bảo toàn tính lồi.

Trong bối cảnh lập kế hoạch chuyển động sử dụng đường cong Bézier, biến liên tục x_v bao hàm cả các điểm điều khiển và thời gian thực hiện của đoạn quỹ đạo. Khi đó, hàm chi phí thể hiện sự đánh đổi giữa thời gian thực thi và công điều khiển (độ mượt). Một dạng thường dùng là tối thiểu hóa năng lượng quỹ đạo (tích phân của bình phương gia tốc) được chuẩn hóa theo

thời gian. Đối với một chuyển tiếp giữa hai đỉnh u và v có thời lượng T_e , chi phí có dạng:

$$\ell_e(x_u, x_v) = \frac{1}{T_e} \int_0^{T_e} \|\ddot{q}(t)\|^2 dt$$

Khi được rời rạc hóa hoặc biểu diễn thông qua các hệ số Bézier, hàm này về bản chất có dạng perspective của biểu thức “bình phương trên tuyến tính” (x^2/y), và do đó là lồi theo cả các điểm điều khiển lẫn thời lượng T_e .

4.3.5 Mô hình Mixed-Integer Convex Programming thông qua Perspective Relaxation

Để chuyển đổi bài toán MINLP không khả giải trực tiếp thành một bài toán Mixed-Integer Convex Program (MICP) có thể giải được, chúng tôi sử dụng một kỹ thuật nói lỏng lồi chặt được gọi là *perspective relaxation*. Cách tiếp cận này cô lập phần không lồi bằng cách nâng các biến liên tục lên một không gian chiều cao hơn và thay thế các ràng buộc song tuyến bằng các ràng buộc bảo toàn luồng tuyến tính trên các biến đã được nâng. Cụ thể, với mỗi cạnh $e = (u, v)$, thay vì sử dụng các biến toàn cục x_u và x_v , ta giới thiệu các biến đại diện cục bộ $z_{e,u}$ và $z_{e,v}$ với $z_{e,u}, z_{e,v} \in \mathbb{R}^n$. Các biến này biểu diễn đóng góp của trạng thái tại các đỉnh u và v tương ứng với việc đi qua cạnh e . Nếu cạnh không được kích hoạt ($y_e = 0$), các đóng góp này bị ép về không:

$$z_{e,u} \in y_e X_u \quad \text{và} \quad z_{e,v} \in y_e X_v$$

tương đương với:

$$\begin{cases} z_{e,u} \in X_u & \text{nếu } y_e = 1 \\ z_{e,u} = 0 & \text{nếu } y_e = 0 \end{cases}$$

Cấu trúc này được hình thức hóa thông qua hàm perspective $\tilde{\ell}_e((z_{e,u}, z_{e,v}), y_e)$, là perspective của hàm chi phí gốc ℓ_e . Một tính chất quan trọng của giải

tích lỗi là: nếu ℓ_e là hàm lỗi, thì perspective của nó cũng lỗi theo tất cả các biến, cho phép sử dụng các bộ giải tối ưu lỗi hiệu quả.

$$\tilde{\ell}_e(z_{e,u}, z_{e,v}, y_e) = \begin{cases} y_e \ell_e\left(\frac{z_{e,u}}{y_e}, \frac{z_{e,v}}{y_e}\right) & \text{nếu } y_e > 0 \\ 0 & \text{nếu } y_e = 0 \text{ và } z_{e,u} = z_{e,v} = 0 \\ +\infty & \text{trường hợp khác} \end{cases}$$

Điểm mấu chốt là, thay vì áp đặt các ràng buộc đẳng thức không lỗi như $z_{e,u} = y_e x_u$, ta mở rộng luật bảo toàn luồng tiêu chuẩn sang các biến liên tục. Luật này yêu cầu rằng, với mọi nút u (trừ nguồn và đích), tổng các trạng thái liên tục đi vào nút thông qua các cạnh vào phải bằng tổng các trạng thái liên tục đi ra thông qua các cạnh ra.

Từ đó, mô hình MICP cho bài toán Đường đi ngắn nhất trong GCS được phát biểu như sau:

$$\text{minimize} \quad \sum_{e=(u,v) \in \mathcal{E}} \tilde{\ell}_e((z_{e,u}, z_{e,v}), y_e) \quad (4.12a)$$

$$\text{subject to} \quad \sum_{e \in \mathcal{E}^+(u)} y_e - \sum_{e \in \mathcal{E}^-(u)} y_e = \delta_u, \quad \forall u \in \mathcal{V} \quad (4.12b)$$

$$\sum_{e \in \mathcal{E}^+(u)} z_{e,u} - \sum_{e \in \mathcal{E}^-(u)} z_{e,v} = \begin{cases} x_s & \text{nếu } u = s \\ -x_t & \text{nếu } u = t \\ 0 & \text{trường hợp khác} \end{cases}, \quad \forall u \in \mathcal{V} \quad (4.12c)$$

$$(z_{e,u}, z_{e,v}) \in y_e \mathcal{X}_e, \quad \forall e \in \mathcal{E} \quad (4.12d)$$

$$z_{e,u} \in y_e X_u, \quad z_{e,v} \in y_e X_v, \quad \forall e = (u, v) \in \mathcal{E} \quad (4.12e)$$

$$y_e \in \{0, 1\}, \quad \forall e \in \mathcal{E} \quad (4.12f)$$

Trong mô hình này, phương trình (4.12b) đảm bảo tính liên thông về mặt

tô-pô (với $\delta_u = 1$ cho nguồn, -1 cho đích và 0 cho các nút còn lại). Phương trình (4.12c) đảm bảo quỹ đạo liên tục và liền mạch trên toàn bộ đồ thị. Các ràng buộc (4.12d) và (4.12e) đảm bảo các biến liên tục nằm trong các tập lồi được co giãn, sao cho nếu $y_e = 0$ thì các biến tương ứng bị ép về gốc tọa độ và không phát sinh chi phí. Mô hình này cung cấp một phép nói lỏng chặt và hiệu quả về mặt tính toán, cho phép bộ giải tìm được các quỹ đạo gần tối ưu toàn cục, đồng thời thỏa mãn cả cấu trúc đồ thị rời rạc lẫn các ràng buộc liên tục phức tạp của bài toán lập kế hoạch chuyển động.

4.4 Proposed solution

4.5 Experiment setup

4.5.1 Implementation details

4.6 Results and analysis

Chapter 5

Conclusion

5.1 Summary

5.2 Future Work

Future research will focus on...

- ...
- ...

References

- [1] R. Deits and R. Tedrake, “Motion planning around obstacles with convex optimization,” *arXiv preprint*, vol. arXiv:2205.04422, 2022.
- [2] Boston Dynamics, *Leaps, bounds, and backflips*, Blog post, 2021.
- [3] Amazon Robotics, *Amazon.com announces first quarter results*, Press release, 2023.
- [4] M. Diehl, H. G. Bock, H. Diedam, and P.-B. Wieber, “Fast direct multiple shooting algorithms for optimal robot control,” in *Fast Motions in Biomechanics and Robotics*, Springer, 2006, pp. 65–93.
- [5] L. E. Kavraki, P. Svestka, J.-C. Latombe, and M. H. Overmars, “Probabilistic roadmaps for path planning in high-dimensional configuration spaces,” *IEEE Transactions on Robotics and Automation*, vol. 12, no. 4, pp. 566–580, 1996.
- [6] S. Boyd and L. Vandenberghe, *Convex Optimization*. Cambridge University Press, 2004.
- [7] R. K. Ahuja, T. L. Magnanti, and J. B. Orlin, *Network Flows: Theory, Algorithms, and Applications*. Prentice Hall, 1993.
- [8] J. M. Lien and N. M. Amato, “Approximate convex decomposition of polygons,” *Computational Geometry: Theory and Applications*, 2006.
- [9] R. Deits and R. Tedrake, “Approximating robot configuration spaces with few convex sets using clique covers of visibility graphs,” *arXiv preprint*, vol. arXiv:2310.02875, 2023.

- [10] R. Deits and R. Tedrake, “Computing large convex regions of obstacle-free space through semidefinite programming,” in *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, 2014.

Appendix A

...

A.1 ...

A.2 ...